

DETERMINING K CLUSTERS IN K-MEANS CLUSTERING WITH THE CRAB AL-
GORITHM

A Thesis
presented to
the Faculty of California Polytechnic State University,
San Luis Obispo

In Partial Fulfillment
of the Requirements for the Degree
Master of Science in Statistics

by
Jasmine Kristine Soriano Cabrera

June 2026

© 2026

Jasmine Kristine Soriano Cabrera

ALL RIGHTS RESERVED

COMMITTEE MEMBERSHIP

TITLE: Determining k Clusters in k-Means Clustering with the
CRAB Algorithm

AUTHOR: Jasmine Kristine Soriano Cabrera

DATE SUBMITTED: June 2026

COMMITTEE CHAIR: Kelly Bodwin, Ph.D.
Professor of Statistics

COMMITTEE MEMBER: Allison Theobald, Ph.D.
Professor of Statistics

COMMITTEE MEMBER: Julia Schedler, Ph.D.
Professor of Statistics

ABSTRACT

Determining k Clusters in k-Means Clustering with the CRAB Algorithm

Jasmine Kristine Soriano Cabrera

Unsupervised clustering often faces the challenge of determining the correct number of clusters in the absence of a true target variable. Traditional methods such as the Elbow Method and the Silhouette Score can produce ambiguous results and rely on assumptions about cluster shape or separation. To address this, we created the Clustering Rivals and Buddies (CRAB) algorithm which evaluates clusters based on stability across multiple subsamples. CRAB uses pairwise classifications to identify points that consistently group together called "Buddies" and points that remain separated called "Rivals". Applied with k-Means, CRAB accurately recovers underlying cluster structures in both spherical and non-spherical datasets, even under varying sample sizes and variances. An accompanying R package implements CRAB with a tidy workflow and easy to follow syntax, making it accessible for research and educational use. By emphasizing stability, CRAB offers a robust alternative for selecting cluster numbers in datasets where traditional methods may fall short.

Keywords: Clustering, Data Science, k-Means.

ACKNOWLEDGMENTS

This paper reflects not only the past five years of my education but also all of the love and support I have received from my family. I would first like to dedicate this to my Mom, my younger sister Kirsten, my younger brother AJ, and most importantly, my Dad, who I miss every day.

I would also like to dedicate this to my second family found in the Statistics Department, where we bonded over many late nights, from the Stat Lab to the Grad Lab.

TABLE OF CONTENTS

	Page
LIST OF TABLES	vii
LIST OF FIGURES	viii
CHAPTER	
1 Introduction	1
1.1 Motivation	1
1.2 Current Methods	2
1.3 Previous Research	5
1.4 CRAB Algorithm	5
2 Properties and Details	7
2.1 Overview	7
2.2 Preparing the Data	8
2.3 Buddies Matrix	9
2.4 Mean Matrix	11
2.5 Similarity Score	12
2.6 CRAB Performance	13
3 The cRab Package in R	18
3.1 Package Goal	18
3.2 Design Philosophy and Ecosystem Integration	18
3.3 Functional Pipeline	19
3.3.1 Phase I: Initialization	19
3.3.2 Phase II: Configuration	20
3.3.3 Phase III: Execution	24
3.4 Computational Strategy and Resource Management	24
3.5 Synthetic Data Generation	26

3.5.1	Point Diagnostics	29
4	Simulation Study	31
4.1	Introduction	31
4.2	Experimental Setup	31
4.3	Spherical Data	32
4.3.1	The Effect of Sample Size (n)	33
4.3.2	The Effect of Variance (σ^2)	34
4.3.3	The Effect of Number of Clusters (k)	37
4.4	Non-Spherical Data	41
4.4.1	Effect of Sample Size	41
4.4.2	Effect of Cluster	47
5	Conclusion	51
	REFERENCES	53
	APPENDICES	
A	buddies_matrix()	55
B	crab()	60
C	crab_bake()	64
D	crab_cols()	67
E	crab_params()	69
F	crab_prep()	72
G	crab_sim_params()	73
H	get_pointvec()	75
I	mean_matrix()	76
J	point_contribution()	79
K	repeat_simulations()	81
L	similarity_score()	86
M	simulate_mvn()	88

LIST OF TABLES

Table	Page
2.1 CRAB Performance on Palmers Penguins Table Line Graph	16
3.1 Execution Stage Example	24
4.1 Win Proportion for Each k Across Different Variances	36
4.2 Cluster k Win Proportion When $k = 2$	38

LIST OF FIGURES

Figure	Page
1.1 Elbow Method Graph using WCSS	2
1.2 Silhouette Score Data	4
1.3 Silhouette Score Graph	4
2.1 CRAB Algorithm	8
2.2 Buddies Matrix Data	10
2.3 Buddies Matrix Example	11
2.4 Mean Matrix Example	12
2.5 Palmers Penguins Data	14
2.6 Elbow Method on Palmers Penguins Data	15
2.7 Silhouette Score on Palmers Penguins Data	16
2.8 CRAB Performance on Palmers Penguins Data	17
3.1 cRab Package Workflow Diagram	19
3.2 crab_prep() Example Output	20
3.3 crab_cols() Example Output	21
3.4 crab_params() Example Output	22
3.5 Configuration Stage Examples	23
3.6 Synthetic Data Generated from simulate_mvn()	26
3.7 Singular Synthetic Dataset Generation with CRAB Algorithm Workflow . .	27
3.8 Multiple Synthetic Dataset Generation with CRAB Algorithm Workflow .	28
3.9 Point Contribution Example	29
4.1 Sample Size Datasets	33
4.2 CRAB Score Across Sample Size	34

4.3	Spherical Clusters by Variance	35
4.4	Multiple Seeds Testing CRAB Score Across Variance	36
4.5	CRAB Scores for Individual and Multiple Seeds When $k = 2$	38
4.6	CRAB Scores for Individual and Multiple Seeds When $k = 3$	39
4.7	CRAB Scores for Individual and Multiple Seeds When $k = 4$	40
4.8	CRAB Scores for Non-Spherical Data with Sample Size = 102	42
4.9	CRAB Scores for Non-Spherical Data with Sample Size = 252	43
4.10	Point Contribution Scores for Sample Size = 252 with $k = 2$	44
4.11	Point Contribution Scores for Sample Size = 252 with $k = 3$	45
4.12	CRAB Scores for Non-Spherical Data with Sample Size = 402	46
4.13	Non-Spherical Data With Different k Values	47
4.14	CRAB Scores for Non-Spherical Data with $k = 2$	48
4.15	CRAB Scores for Non-Spherical Data with $k = 3$	49
4.16	CRAB Scores for Non-Spherical Data with $k = 4$	50

Chapter 1

INTRODUCTION

1.1 Motivation

Unsupervised learning is a type of machine learning where data is unlabeled and the true target variable is unknown. Practitioners often use it to identify groups of observations, reduce dimensionality, or detect anomalies within data. While these methods can reveal meaningful structure in data, evaluating their results is inherently difficult because no ground truth exists, unlike in supervised learning.

Clustering is the most widely recognized form of unsupervised learning, with K-means being one of its most commonly used algorithms. K-means starts by initializing k centroids in the data space and assigns each point to the nearest centroid based on Euclidean distance. It then updates the centroids as the mean of their assigned points and repeats this process until the cluster assignments stabilize. Despite its simplicity, K-means has notable limitations. One flaw is the final clustering depends heavily on the initial placements of the centroids (Jaeger & Banks, 2023), which can lead to inconsistent or suboptimal results. Additionally, the use of Euclidean distance assumes that the clusters themselves are roughly spherical, an assumption that often fails for real-world data.

A major challenge in clustering as a whole is determining the appropriate number of clusters. Without prior knowledge, this choice can significantly affect the outcome, and different methods often suggest different answers. Two of the most commonly used techniques are the Elbow Method and the Silhouette Score.

1.2 Current Methods

The Elbow Method is a visual approach that evaluates clustering by calculating the within-cluster sum of squares (WCSS). This measure sums the squared Euclidean distances between each point and its assigned cluster centroid for different values of k . As the number of clusters increases, the WCSS naturally decreases because smaller clusters reduce within-cluster variation. Analysts typically identify the optimal number of clusters at the point where the decrease in WCSS slows dramatically and begins to level off, forming what is known as the “elbow”. They then choose the value of k just before this plateau as the correct number of clusters. Although this method works well when the elbow is clearly visible, it does not always provide a clear answer (Kodinariya & Makwana, 2013).

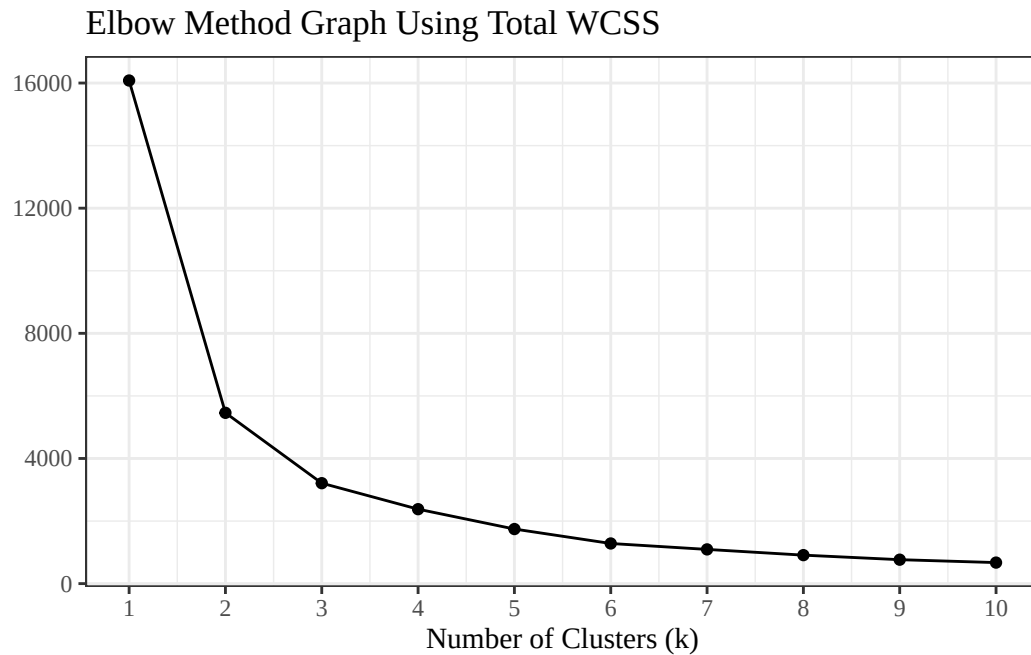


Figure 1.1: Elbow Method Graph using WCSS

Figure 1.1 illustrates a case where WCSS decreases gradually for all values of k , making it difficult to identify a distinct elbow. Without a clear plateau to guide the decision, users must select the optimal number of clusters subjectively, which can produce very different results depending on the chosen value of k .

In contrast to the Elbow Method, the Silhouette Score provides a more quantitative approach by removing the subjectivity of visual interpretation. Instead of relying on a plot, it judges clusters using a mathematical measure and selects the number of clusters based on the global maximum which corresponds to well separated and cohesive clusters. However, this method introduces its own limitation due to the assumptions it makes about cluster structure.

The Silhouette Score is defined as:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where $a(i)$ is the average distance between point i and all other points in the same cluster, and $b(i)$ is the average distance between point i and points in the nearest neighboring cluster.

Intuitively, this measure evaluates how well a point fits within its assigned cluster compared to others. Because of its formulation, the Silhouette Score favors compact and well separated clusters. As a result, it performs well when clusters are roughly uniform in shape, but struggles with oddly shaped or elongated structures that real data tends to resemble. Additionally, maximizing the global score can bias the method toward a solution with fewer clusters, prioritizing separation over accurately capturing the underlying structure. This limitation is illustrated in Figures 1.2 and 1.3.

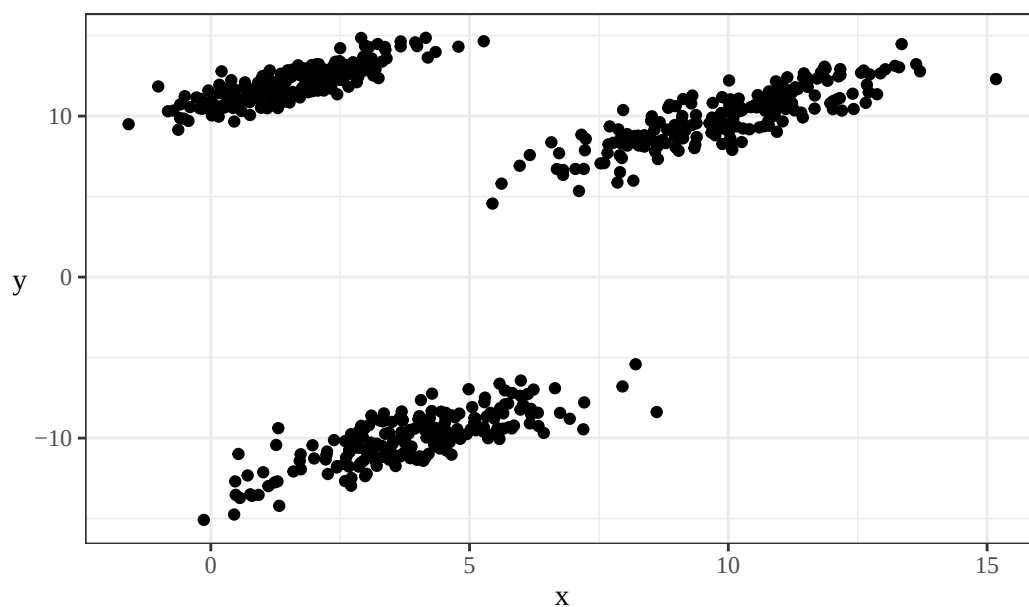


Figure 1.2: Silhouette Score Data

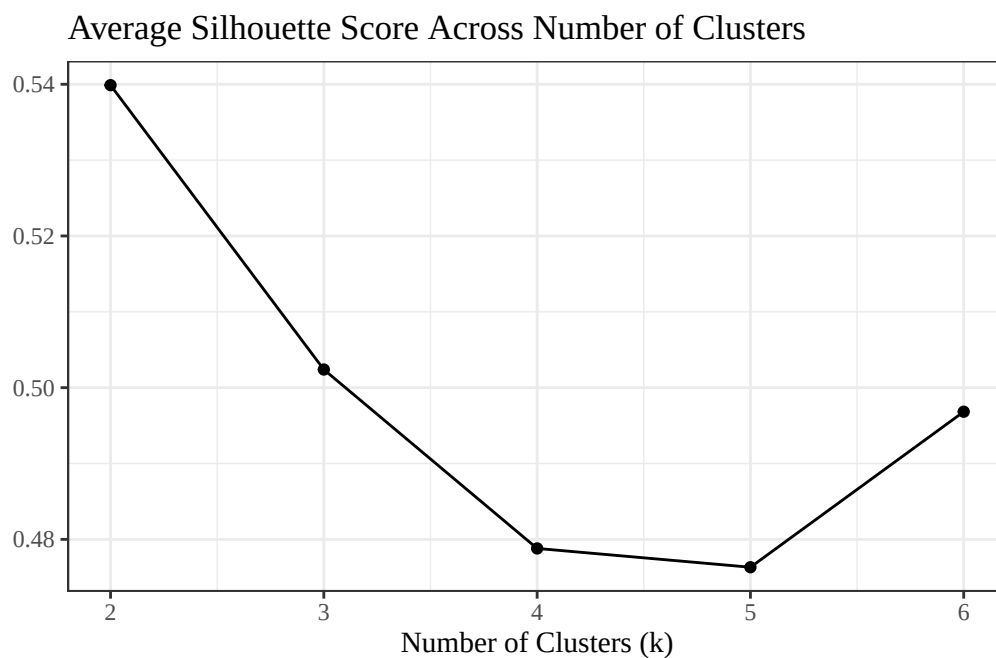


Figure 1.3: Silhouette Score Graph

Although the dataset visually contains three distinct clusters, the Silhouette Score identifies $k = 2$ as the most optimal. This occurs because the two upper clusters are relatively close to

each other compared to the lower cluster, leading the algorithm to favor greater separation over correctly identifying all three groups.

1.3 Previous Research

These drawbacks have motivated alternative approaches that go beyond evaluating clusters based solely on distance or compactness. One method introduced by Asa Ben-Hur and colleagues, evaluates clustering through stability (Ben-Hur, Elisseeff, & Guyon, 2002). Their approach stems from the idea that a clustering into k groups captures the inherent structure if similar cluster assignments are obtained across various subsamples of the data. To assess this, the method repeatedly clusters subsets of the data using a hierarchical clustering method and compares the consistency of assignments between observations found in both subsets. This method measures similarity between clusters using cosine similarity, producing a distribution of similarity scores for each value of k . When a true underlying structure exists, these scores remain consistently high and concentrated near 1, indicating that the clustering is stable across perturbations of the data.

1.4 CRAB Algorithm

Building on this idea, the CRAB algorithm evaluates clustering quality through stability rather than relying on assumptions about cluster shape or separation. It repeatedly subsamples and reclusters the data to measure how consistently pairwise observations are assigned to the same cluster across different samples. This approach shifts the focus from how the clusters look visually to their reproducibility, providing stronger evidence that the identified groupings are meaningful.

Unlike the Elbow Method and Silhouette Score, the CRAB algorithm makes no assumptions about the data or group structure, allowing it to better capture more complex and irregular patterns. Additionally, its use of a global minimum provides a clear and objective selection

criterion, avoiding the ambiguity of visual interpretation and the bias toward compact clusters. Although this approach may require additional computation due to repeated sampling, it offers a more reliable way to determine the number of clusters.

The following chapters expand on this method. Chapter 2 provides a more detailed mathematical explanation, while Chapter 3 introduces an R package with functions that demonstrate the capabilities and applications of the cRab algorithm on both real and simulated data.

Chapter 2

PROPERTIES AND DETAILS

2.1 Overview

The CRAB algorithm, which stands for Clustering Rivals and Buddies, is designed to evaluate clustering performance by examining how consistently points are grouped together across multiple subsamples of the data. Unlike traditional methods that rely on assumptions about cluster shape, size, or separation, CRAB emphasizes reproducibility, providing a more robust measure. At its core, CRAB focuses on pairwise relationships between observations. By tracking how often two points end up in the same cluster across repeated resampling, the algorithm quantifies the stability of cluster assignments. This approach allows it to handle irregular clusters that would challenge methods such as the Elbow Method and Silhouette Score. The output is a single metric, the CRAB score, which provides a clear answer for selecting the most appropriate number of clusters while minimizing subjectivity.

cRab Algorithm:

Inputs:

data: Dataset to analyze
min k: Starting number of clusters k to evaluate
max k: Ending number of clusters k to evaluate
subsample_prop: Fraction of the dataset to use for each resample
num_subsamples: Number of resamples to generate
cluster_method: Clustering algorithm to apply (i.e. K-means)

Output: A data frame where each row corresponds to a tested k cluster. The data frame includes the CRAB score, runtime, and a "Best Cluster" column. Only the row corresponding to the lowest CRAB score is marked as "BEST" while all other rows remain blank.

For all desired values of k :

1. **Subsample the data:** Randomly select f proportion of the observations.
 2. **Cluster the subsample:** Apply the chosen clustering method to assign cluster labels to each point in subsample.
 3. **Record pairwise assignments:** Assign Buddies and Rivals for each pair.
 - **Buddies (+1):** In the same cluster
 - **Rivals (-1):** In different clusters
 - **Ignore (0):** One or both are not in subsample
 4. **Repeat resampling and clustering:** Perform steps 1-3 for a total of r resamples, generating a Buddies Matrix for each iteration.
 5. **Aggregate pairwise stability:** Combine the Buddies Matrices into a Mean Matrix to measure how consistently each pair of points clusters together across resamples.
 6. **Compute CRAB score:** Calculate the average squared distance from 1 across all valid Mean Matrix entries to produce a single cluster stability metric.
 7. **Select optimal k:** Choose k with the lowest CRAB score as the most stable clustering solution.
-

Figure 2.1: CRAB Algorithm

2.2 Preparing the Data

Before applying the CRAB algorithm, ensure that the dataset is preprocessed appropriately for the chosen clustering method. For example, if using K-means, this may include handling missing values and standardizing features so that the variables on different scales do not

disproportionately influence the clustering algorithm.

The CRAB algorithm requires three main inputs:

- **Subsample Proportion (f):** Fraction of the dataset to use for each resample
- **Number of resamples (r):** Number of resamples to generate (i.e. number of times the data will be subsampled)
- **Range of k values:** The different cluster counts to evaluate

For each resample, CRAB draws a random subset of the data containing f proportion of the observations and applies the chosen clustering method (i.e. K-means) to assign cluster labels to each point. By repeating this process across multiple resamples, the algorithm detects which cluster assignments are consistently reproduced, instead of placing confidence on any single clustering of the full dataset.

2.3 Buddies Matrix

After clustering each resample, CRAB constructs a Buddies Matrix, B , to track pairwise relationships between observations. This $n \times n$ matrix encodes whether points i and j were assigned to the same cluster:

$$B_{i,j} = \begin{cases} 1, & \text{if they are in the same cluster} \\ -1, & \text{if they are in different clusters} \\ 0, & \text{if one or both points are not included in the subsample} \end{cases}$$

To reduce computation and take advantage of symmetry, CRAB sets the diagonal and lower triangle of each Buddies Matrix to NA. Each resample produces one Buddies Matrix, resulting in r matrices in total. Collectively, these matrices capture how pairwise relationships vary across resamples, forming the foundation for measuring cluster stability. Ideally, each pair

of observations (i, j) should consistently have either 1s (always assigned to the same cluster) or -1s (always assigned to different clusters) across resamples. Pairs that switch between 1 and -1 indicate instability in the clustering assignments.

For example, Figure 2.1 shows a random sample from a mock dataset clustered using K-means. The five labeled points illustrate all possible scenarios: Observations 1 and 3 belong in different clusters, while Observations 4 and 5 are in the same cluster. Observation 2, although part of the original dataset, was not included in this particular sample.

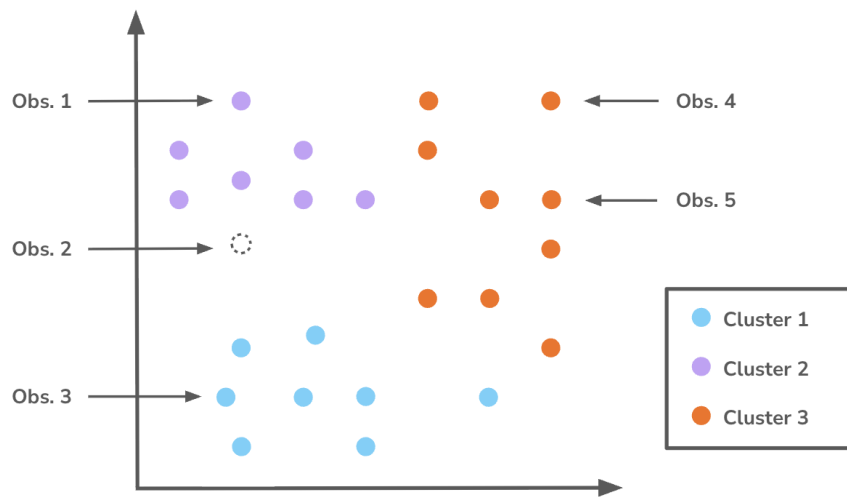


Figure 2.2: Buddies Matrix Data

Figure 2.2 shows how these pairwise relationships are represented in a Buddies Matrix.

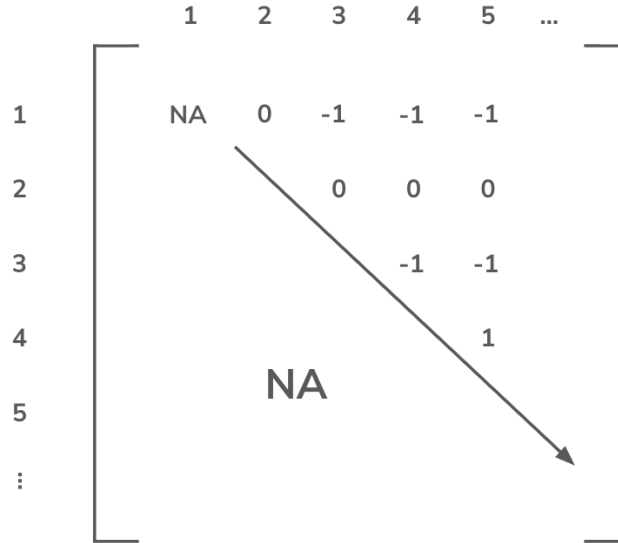


Figure 2.3: Buddies Matrix Example

2.4 Mean Matrix

The algorithm aggregates the Buddies Matrices to form the Mean Matrix, which summarizes the overall stability of pairwise cluster assignments. For each entry (i, j) , it first counts the number of resamples where both points were included. It then sums the corresponding entries from those Buddies Matrices and divides by the number of comparisons to obtain the mean. Each entry ranges from -1 and 1, reflecting how consistently the pair of points clusters together across resamples. Values closer to -1 suggest that the points are often placed in different clusters, while values closer to 1 indicate that the points are frequently assigned to the same cluster. Entries near 0 indicate inconsistent or ambiguous pairwise assignments. The Mean Matrix acts as a stability map, revealing which relationships remain robust across different subsamples of the data. Figure 2.3 visually demonstrates how the algorithm constructs the Mean Matrix from the Buddies Matrices.

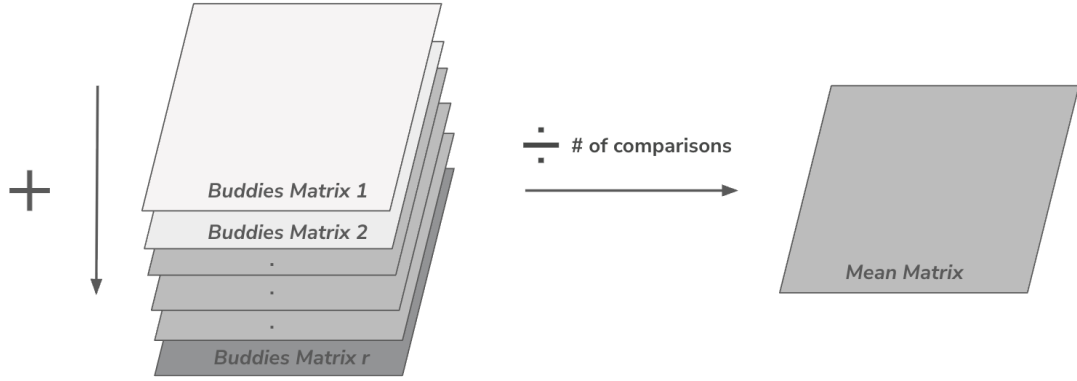


Figure 2.4: Mean Matrix Example

2.5 Similarity Score

To summarize the information in the Mean Matrix, the CRAB score quantifies cluster stability. Let M represent the Mean Matrix with entries $M_{i,j}$ for each pair of observations. The CRAB score is defined as:

$$\text{CRAB Score} = \frac{1}{|V|} \sum_{(i,j) \in V} (1 - |M_{i,j}|)^2$$

Where:

- V is the set of all valid entries in the vectorized Mean Matrix (i.e. all pairs that are not NA)
- $|V|$ is the number of valid entries
- $M_{i,j}$ is the Mean Matrix entry for the i, j pair

Scores close to 0 indicate that points consistently group together across all subsamples, suggesting highly reproducible clusters. In contrast, higher scores denote greater instability, signaling that cluster assignments vary significantly depending on the subsample. The CRAB score condenses these various pairwise relationships into a single metric that can be

compared across different values of k .

2.6 CRAB Performance

By computing the CRAB scores across different values of k , the algorithm identifies the most stable clustering as the one with the lowest score. This provides an objective criterion for selecting the number of clusters. In many cases, when visualized across increasing k , the optimal value appears as a “pinching point” in the plot, where the score decreases before rising again for larger k values.

To illustrate the performance of CRAB, we use the well-known Palmer Penguins dataset (Horst, Hill, & Gorman, 2020) as an effective benchmark. This dataset is particularly useful because it contains a true class label, species, allowing for direct comparison between identified clusters and the actual groupings. There are three unique species along with several physical measurements, such as bill length, bill depth, and flipper length. For this analysis, we select flipper length and bill length, as these features provide strong separation between species with minimal overlap among observations.

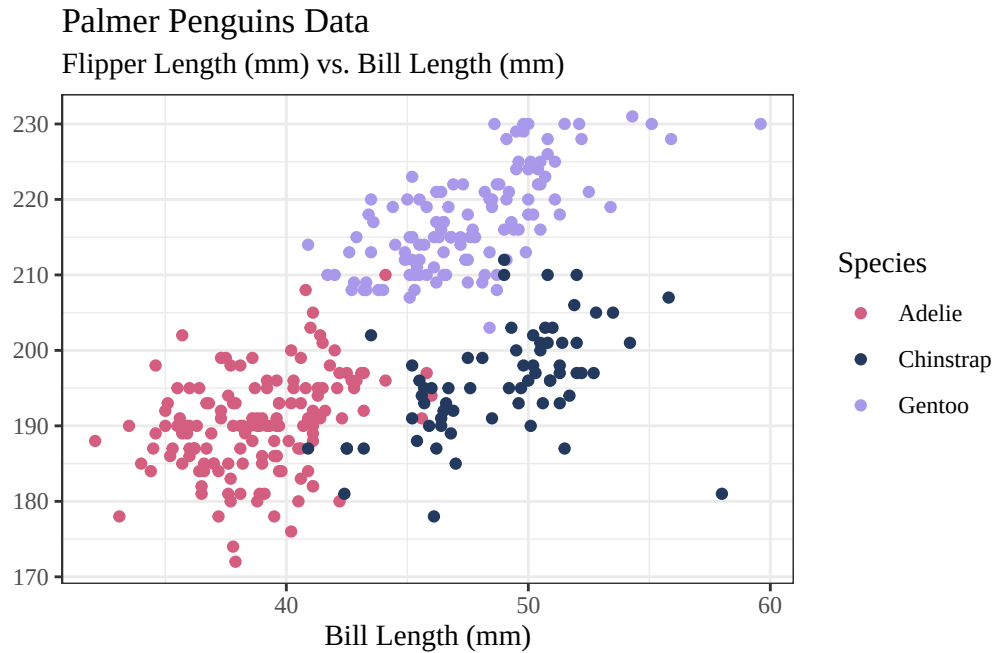


Figure 2.5: Palmers Penguins Data

When applying the Elbow Method, the resulting plot in Figure 2.5 shows a gradual decrease in WCSS rather than a distinct bend. As discussed previously, this makes it difficult to clearly identify a clear optimal number of clusters, since there is no obvious point where the reduction in error slows significantly. As a result, the method provides little guidance and requires subjective interpretation.

Elbow Method Graph Using Total WCSS
On Palmers Penguins Data

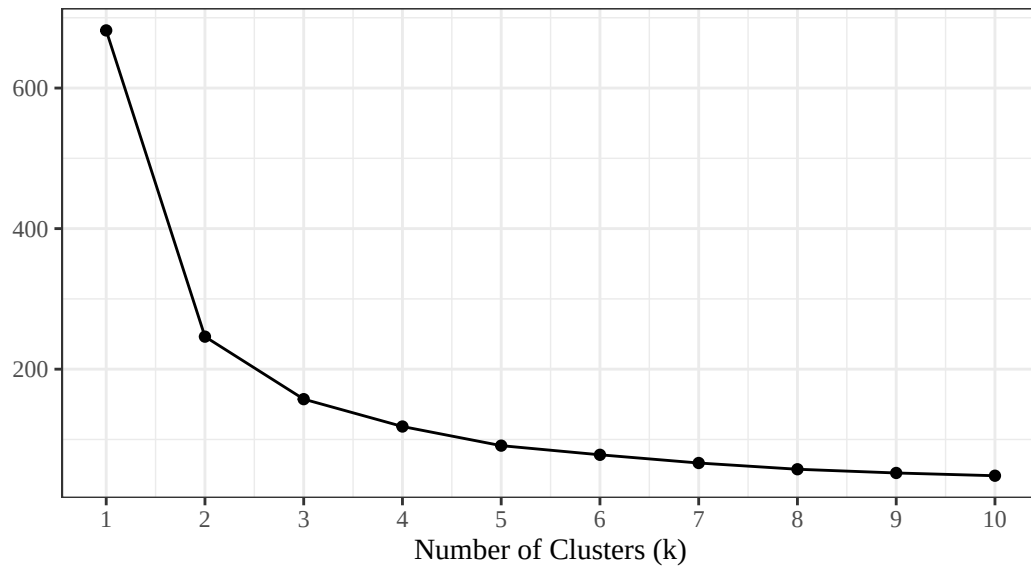


Figure 2.6: Elbow Method on Palmers Penguins Data

The Silhouette Score, shown in Figure 2.6, suggests that two clusters are optimal, as this value produces the highest average silhouette score. According to this metric, the data is best partitioned into two groups. However, this result conflicts with the known structure of the dataset, which contains three distinct species. This discrepancy highlights the method's tendency to favor solutions that maximize separation.

Average Silhouette Score Across Number of Clusters
On Palmers Penguins

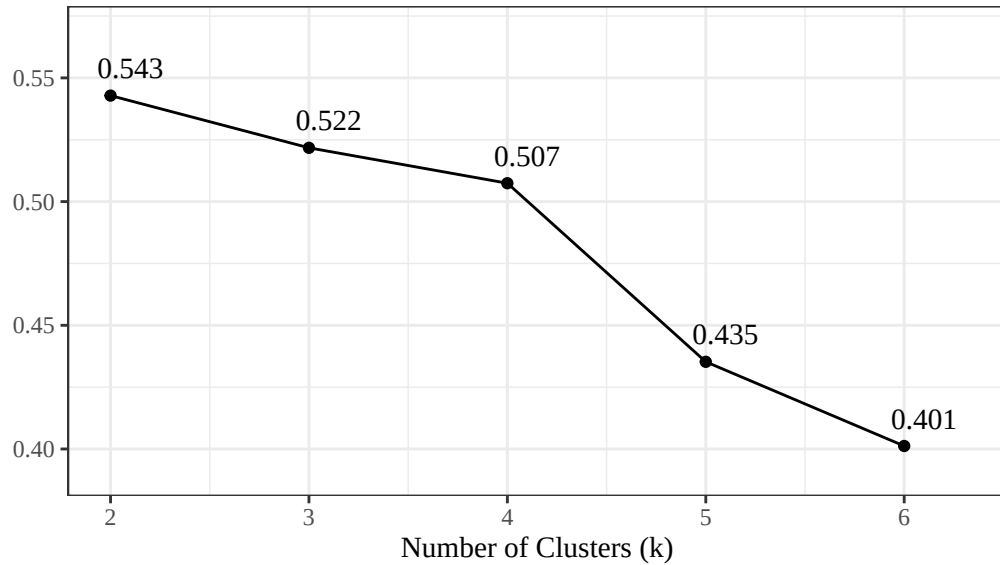


Figure 2.7: Silhouette Score on Palmers Penguins Data

In contrast, the CRAB Score correctly identifies three clusters as the optimal solution with a score of 0.0048. The next lowest score occurs at $k = 2$, which is consistent with the alternative methods. Although the “pinching point” appears less visually pronounced in this example, the local minimum at $k = 3$ provides a clear and objective indication of the suggested number of groupings. More importantly, this result aligns with the known structure of the dataset, as defined by the species label.

Table 2.1: CRAB Performance on Palmers Penguins Table Line Graph

k	Score	Time (seconds)	Best Cluster (k)
2	0.0073200	52.255	
3	0.0048351	48.618	BEST
4	0.1260321	46.879	
5	0.0932065	42.886	
6	0.0937262	36.619	

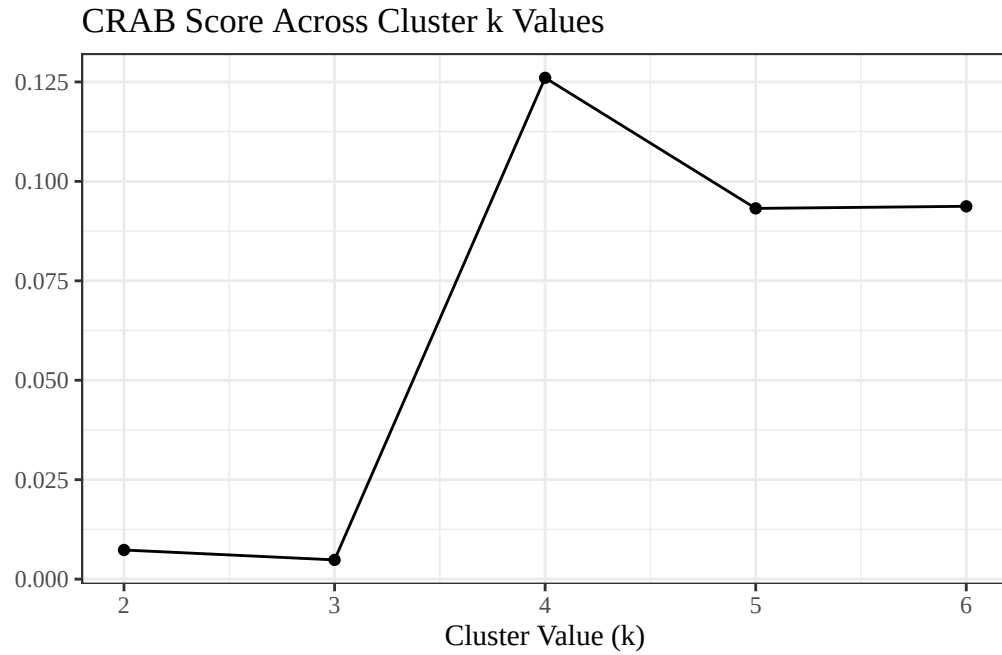


Figure 2.8: CRAB Performance on Palmers Penguins Data

Chapter 4 goes more into depth through a series of simulations that demonstrate CRAB's performance across various datasets, highlighting its ability to identify the true structure of the data. The chapter uses boxplots and other visualizations to illustrate how CRAB performs.

Chapter 3

THE CRAB PACKAGE IN R

3.1 Package Goal

After introducing the CRAB algorithm and outlining its step-by-step logic, we now turn to its practical implementation in R. The cRab package provides a reliable, automated framework for calculating the CRAB score, emphasizing a tidy and efficient workflow that aligns seamlessly with the tools and conventions R users already rely on. By formalizing this process into an accessible package, cRab offers a rigorous yet intuitive toolkit for validating cluster structures without the need for complex, or custom syntax.

3.2 Design Philosophy and Ecosystem Integration

The cRab package extends the tidyclust ecosystem (Hvitfeldt & Bodwin, 2025) and builds on the core principles of the tidymodels (Kuhn & Wickham, 2020) framework. Rather than being a standalone procedure, the package treats clustering as an integrated step of a machine learning workflow. This architectural choice keeps the package familiar to modern data scientists by relying exclusively on standard R objects like tibbles and parsnip-style models. By adopting this “recipe” style, cRab fits directly into existing data pipelines without requiring users to learn new data formats.

The core logic of cRab uses the native R pipe (`|>`), which establishes a baseline requirement of R Version 4.1. While this introduces a specific version constraint, it prioritizes the long-term stability and readability of the codebase. By utilizing the native pipe, the package reduces external dependencies, such as the magrittr package (Bache & Wickham, 2025) which houses the magrittr operator (`%>%`). As a result, the package maintains a lighter installation

footprint and produces more transparent error messages during debugging. Together, these design choices promote consistent and easy to follow syntax as the R language continues to evolve.

3.3 Functional Pipeline

The cRab package architecture leverages a workflow that preserves the original data and builds a working object that updates as the analysis progresses. Unlike traditional R functions that return immediate transformations of data, cRab employs a system that structures the analysis into three logical phases: Initialization, Configuration, and Execution (Figure 3.1).

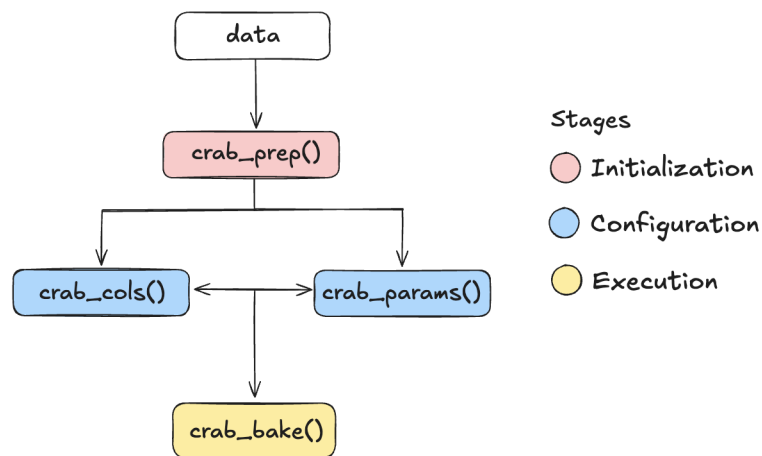


Figure 3.1: cRab Package Workflow Diagram

3.3.1 Phase I: Initialization

The pipeline begins with the `crab_prep` object, which serves as the central container for the entire validation process. Upon initialization, the package generates a metadata-rich list that stores the raw data alongside a set of empty slots reserved for variable selection and hyperparameters. By storing information within this object, cRab facilitates a precomputation phase where it evaluates and records the structure of the data. This setup keeps later stages of the pipeline computationally efficient and consistent by avoiding repeated computation

of the same information. To visualize the workflow setup, the iris dataset (R Core Team, 2026) will be used as an example.

```
iris |>
  crab_prep()

  $data

  $original_cols
[1] "Sepal.Length" "Sepal.Width" "Petal.Length" "Petal.Width" "Species"

  $specified_cols
NULL

  $specified_params
list()
```

Figure 3.2: crab_prep() Example Output

3.3.2 Phase II: Configuration

After initialization, the workflow enters the Configuration Stage. To improve flexibility and readability, cRab splits configuration into two modular functions: `crab_cols()` and `crab_params()`. The first function, `crab_cols()`, manages variable selection, allowing the user to define which features contribute to distance metrics and clustering stability. It populates these choices in the `specified_cols` element within the `crab_prep` object (Figure 3.3).

```
iris |>
  crab_prep() |>
  crab_cols(c("Petal.Length", "Petal.Width"))
```

```
$data

$original_cols
[1] "Sepal.Length" "Sepal.Width" "Petal.Length" "Petal.Width" "Species"

$specified_cols
[1] "Petal.Length" "Petal.Width"

$specified_params
list()
```

Figure 3.3: `crab_cols()` Example Output

Complementing this, `crab_params()` defines the hyperparameters required for the Rivals and Buddies logic, including subsampling rates and number of iterations. This step updates the `specified_params` slot (Figure 3.4).

```
iris |>
  crab_prep() |>
  crab_params(min_k = 3, max_k = 5, num_subsamples = 10,
             cluster_method = "kmeans")
```

```

$data

$original_cols
[1] "Sepal.Length" "Sepal.Width" "Petal.Length" "Petal.Width" "Species"

$specified_cols
NULL

$specified_params
$specified_params$min_k
[1] 3

$specified_params$max_k
[1] 5

$specified_params$num_subsamples
[1] 10

$specified_params$subsample_prop
[1] 0.8

$specified_params$cluster_method
[1] "kmeans"

```

Figure 3.4: `crab_params()` Example Output

These two functions operate independently and can be input in any order. As a result, a researcher can call `crab_cols()` before or after `crab_params()`, and overwrite previous settings without affecting the underlying data. Each call made updates only its designated slot within the `crab_prep` object, keeping the workflow modular and transparent.

```

# option 1

iris |>

  crab_prep() |>

  crab_cols(c("Petal.Length", "Petal.Width")) |>

  crab_params(min_k = 3, max_k = 5, num_subsamples = 10,
              cluster_method = "kmeans")

# option 2

```



```
iris |>
  crab_prep() |>
  crab_params(min_k = 3, max_k = 5, num_subsamples = 10,
              cluster_method = "kmeans") |>
  crab_cols(c("Petal.Length", "Petal.Width"))

$data
$original_cols
[1] "Sepal.Length" "Sepal.Width" "Petal.Length" "Petal.Width" "Species"

$specified_cols
[1] "Petal.Length" "Petal.Width"

$specified_params
$specified_params$min_k
[1] 3

$specified_params$max_k
[1] 5

$specified_params$num_subsamples
[1] 10

$specified_params$subsample_prop
[1] 0.8

$specified_params$cluster_method
[1] "kmeans"
```

Figure 3.5: Configuration Stage Examples

Before moving to the computationally intensive Execution phase, cRab implements a rigorous validation layer. Each function checks that the input object contains the required metadata and valid data types. The package also provides educational warnings to support correct usage. For instance, if a user sets $k = 1$, a scenario that is mathematically trivial for clustering stability, the package does not stop the execution with an error. Instead, it issues a warning that $k = 1$ does not produce meaningful cluster results. This approach encourages more informed choices while maintaining workflow continuity.

3.3.3 Phase III: Execution

The workflow transitions from specifications to results exclusively during the Execution Stage via `crab_bake()`. At this phase, the function reconciles the stored metadata and applies the configured logic to the raw data. It undoubtedly handles the heavy lifting by performing iterative subsampling and calculation of the final CRAB scores. These computations return a tidy data frame that summarizes cluster stability in a clear and reproducible format.

```
iris |>
  crab_prep() |>
  crab_cols(c("Petal.Length", "Petal.Width")) |>
  crab_params(min_k = 3, max_k = 5, num_subsamples = 10,
              cluster_method = "kmeans") |>
  crab_bake()
```

Table 3.1: Execution Stage Example

k	Score	Time (seconds)	Best Cluster (k)
2	0.0001581	21.969	BEST
3	0.0002594	44.644	
4	0.0941276	21.463	
5	0.1022005	20.908	

3.4 Computational Strategy and Resource Management

The cRab pipeline relies on Buddies Matrices with an $n \times n$ structure, which causes the computational cost of cluster validation to grow quadratically as the dataset size increases. To maintain performance across different machines, the package maximizes throughput through parallelism while limiting memory usage in its core data structures.

To accelerate the iterative Rivals and Buddies computations, cRab utilizes the doParallel (Corporation & Weston, 2022) and foreach (Microsoft & Weston, 2022) libraries. Parallelization significantly reduces the total execution time by distributing iterations across multiple CPU cores, but consequently also increases memory usage since each core requires a copy of the global environment.

To prevent system overload, the package applies a conservative core reservation strategy by default:

$$\text{Cores}_{\text{Active}} = \max(1, \text{detectCores}() - 5)$$

By reserving several cores for background processes, the system remains responsive during execution.

The primary computational bottleneck arises during the construction of the Buddies Matrix, which has a time and space complexity of $O(n^2)$, where n represents the number of observations in each subsample. As n increases, storing all pairwise relationships can exceed available RAM and lead to memory errors.

To address this, cRab uses a more efficient storage strategy. The package reduces redundancy by storing only unique relationships. By utilizing the symmetry of the matrix and keeping only the upper triangle of indices, it effectively cuts storage requirements roughly in half. Additionally, the package applies sparse indexing, tracking only relationships between observations present in each subsample.

These optimizations reduce memory demands and allow larger datasets to run on standard hardware without sacrificing the accuracy of the CRAB score.

3.5 Synthetic Data Generation

While cRab is designed to validate clusters in real-world datasets, controlled simulations provide a way to evaluate its sensitivity when the true cluster structure is known. To support this, the package includes `simulate_mvn()`, a utility that allows users to benchmark the CRAB score on datasets with predefined cluster properties before applying the tool to ambiguous or unlabeled data.

The simulation framework uses Multivariate Normal (MVN) distributions from the MASS package (Venables & Ripley, 2002) to offer precise control over cluster separation, orientation, and density. By manipulating the covariance structure, users can introduce varying levels of overlap and geometric distortion, creating a range of clustering scenarios.

The `simulate_mvn()` function generates data using three primary user inputs: the total number of observations (n), a variance scalar (σ^2), and the desired number of clusters (k). These parameters allow users to control how distinct or overlapping the clusters appear.

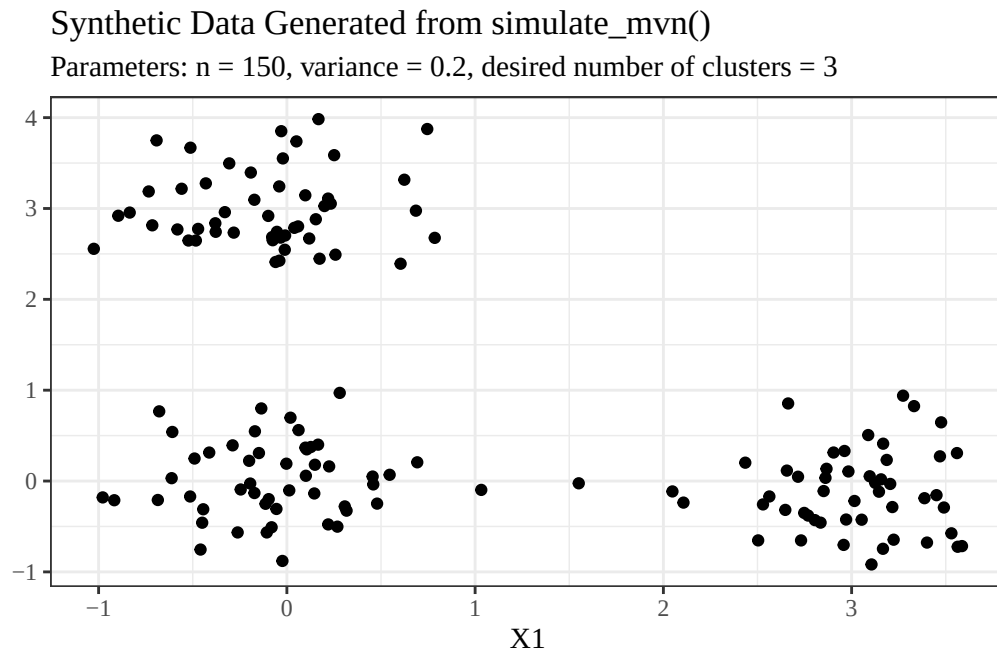


Figure 3.6: Synthetic Data Generated from `simulate_mvn()`

For a single simulated dataset, the workflow follows the standard pipeline (Figure 3.7). In this scenario, `simulate_mvn()` generates the data, which then passes through standard preprocessing and parameter specification layers before applying the CRAB algorithm.

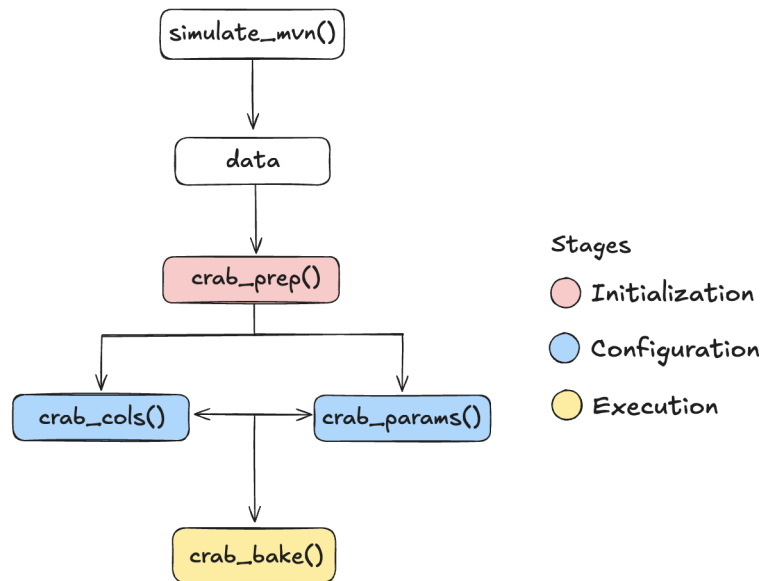


Figure 3.7: Singular Synthetic Dataset Generation with CRAB Algorithm Workflow

Below is an example code chunk:

```
# data from simulate_mvn()
data |>
  crab_prep() |>
  crab_cols(c("X1", "X2")) |>
  crab_params(min_k = 2, max_k = 5, num_subsamples = 10,
              cluster_method = "kmeans") |>
  crab_bake()
```

When evaluating performance across multiple runs or varying noise levels, the workflow shifts to a simulation focused approach (Figure 3.8). One option mirrors the standard pipeline by initializing with `crab_prep(NULL)` and specifying generative parameters through

`crab_sim_params()`, such as the number of runs, variance, and sample size. The `repeat_simulations()` function then replaces `crab_bake()` and applies the CRAB algorithm across multiple simulated datasets with consistent settings. Alternatively, users can bypass the pipeline entirely and pass all parameters into `repeat_simulations()` directly.

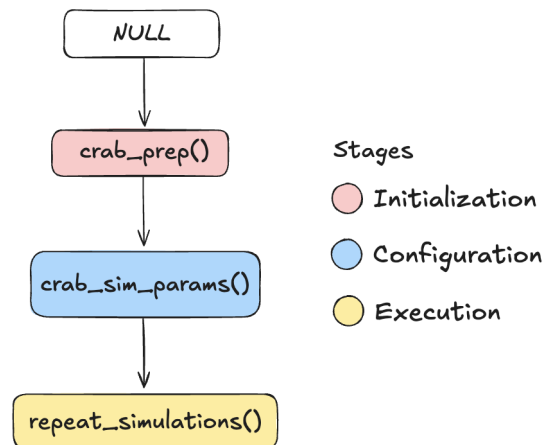


Figure 3.8: Multiple Synthetic Dataset Generation with CRAB Algorithm Workflow

To illustrate these options, consider the following implementations:

```
# option 1 -- pipeline format
crab_prep(data = NULL) |>
  crab_sim_params(min_k = 2, max_k = 5, num_subsamples = 10,
                  num_runs = 2, variance = c(0.3, 0.4),
                  cluster_method = "kmeans") |>
  repeat_simulations()

# option 2 -- directly passing in parameters
repeat_simulations(min_k = 2, max_k = 5, num_subsamples = 10,
                  num_runs = 2, variance = c(0.3, 0.4),
                  cluster_method = "kmeans")
```

Chapter 4 explores the results of these simulations in greater detail.

3.5.1 Point Diagnostics

In addition to providing a global measure of cluster stability, the cRab framework includes diagnostic tools to identify which observations drive that stability or, more importantly, its instability. The `point contribution()` function allows for a granular, observation-level analysis by quantifying and visualizing how each individual data point impacts the final CRAB score.

In practice, raw instability values are difficult to interpret. Although they are mathematically meaningful, their scale often varies across different datasets and dimensionalities, making visualization hard to read. To improve interpretability, `point_contribution()` applies a sigmoid transformation that maps contributions onto a bounded $[0, 1]$ scale. This transformation standardizes the values and makes relative differences easier to interpret.

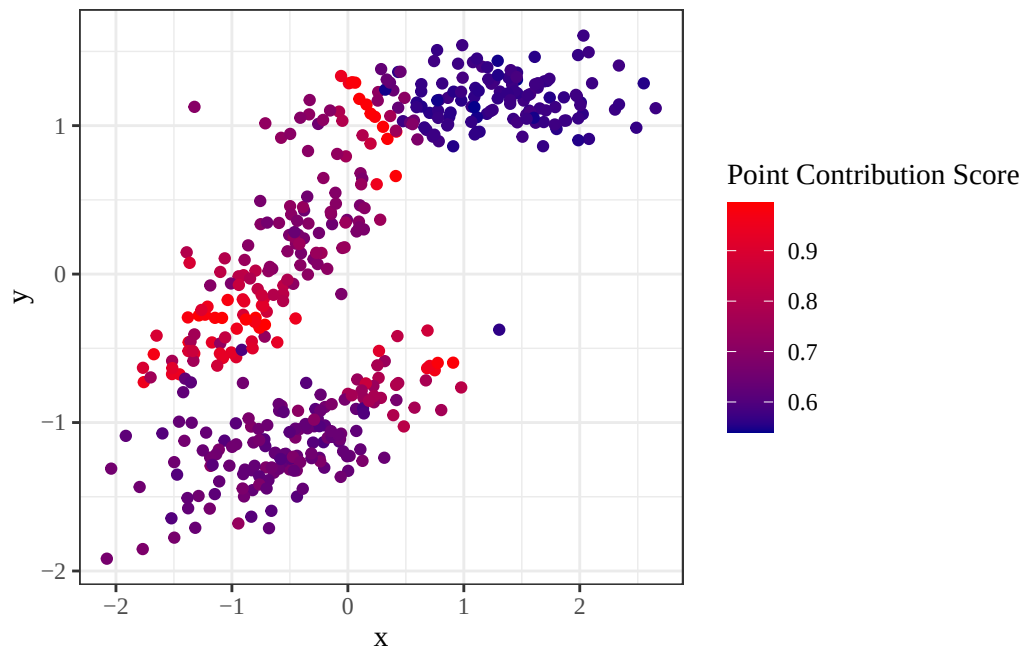


Figure 3.9: Point Contribution Example

Observations that act as “swing voters” in the clustering structure are typically located

near cluster boundaries, making them the most likely to switch cluster assignments across subsamples. Figure 3.9 illustrates this behavior where high contribution points appear as bright red regions concentrated near decision boundaries. In contrast, observations that don't change between clusters are labeled in a dark blue color and have a contribution score closer to 0. By isolating these observations, the function helps identify which data points disproportionately influence the CRAB score.

Chapter 4

SIMULATION STUDY

4.1 Introduction

This study evaluates the performance and reliability of the CRAB algorithm under controlled conditions. Our primary goal is to establish a clear performance baseline by measuring how the CRAB score responds to changes in data structure and cluster complexity.

The analysis focuses on two distinct cluster geometries: spherical and non-spherical. For spherical clusters, we manipulate sample size (n), cluster variance, and the number of clusters (k), whereas for non-spherical clusters, only the sample size and number of clusters are varied. Together, these dimensions allow us to evaluate how the CRAB score behaves across a range of conditions, from well separated and minimal noise settings to more ambiguous and complex scenarios.

4.2 Experimental Setup

For spherical clusters, we generated data using the `simulate_mvn()` function which calls the MASS package. This function produces MVN distributions with user-specified variances, allowing precise control over cluster overlap.

Non-spherical clusters were created manually, one cluster at a time, also using functions from the MASS package. This method produces elongated or irregular cluster shapes, offering a more challenging test for the CRAB algorithm.

The simulation grid varies three primary variables:

- **Sample Size (n):** $n \in \{102, 252, 402\}$ representing small, moderate and larger

datasets.

- **Variance (σ^2):** $\sigma^2 \in \{0.2, 0.5, 0.8, 1\}$ controlling cluster compactness and overlap
- **Number of Clusters (k):** $k \in \{2, 3, 4\}$ to evaluate performance across total cluster groupings.

Each parameter combination isolates one factor at a time while holding the others fixed. This design ensures that changes in CRAB performance can be directly attributed to the variable being tested.

Performance is evaluated based on the algorithm’s ability to correctly identify the true number of clusters. Strong results occur when the CRAB score reaches its global minimum at the correct value of k , indicating a clear and accurate detection of cluster structure. The shape of the score curve provides additional insight, as a clear “pinching point” suggests a well defined clustering solution.

To account for randomness, each scenario was repeated 25 times using the `repeat_simulations()` function. This produces a distribution of CRAB scores rather than a single outcome, allowing for a more reliable assessment of stability. Consistent results across runs indicate robust performance, while greater variability reflects sensitivity to noise or overlap.

4.3 Spherical Data

To assess CRAB’s behavior on spherical clusters, we conducted experiments by systematically varying sample size, cluster variance, and the number of clusters. This setup allows us to observe how tightly grouped versus overlapping clusters influence the algorithm’s ability to detect the correct structure.

Visualization and summary statistics from these simulations highlight trends in CRAB performance, showing how changes in cluster compactness and size affect score distributions and CRAB’s reliability.

4.3.1 The Effect of Sample Size (n)

Sample size can significantly influence CRAB's performance, affecting factors such as the proportion of the sample used and computation time. To evaluate its impact, we tested CRAB on datasets with sample sizes of 102, 252, and 402, while keeping the number of clusters fixed at $k = 3$ and the variance at 0.5.

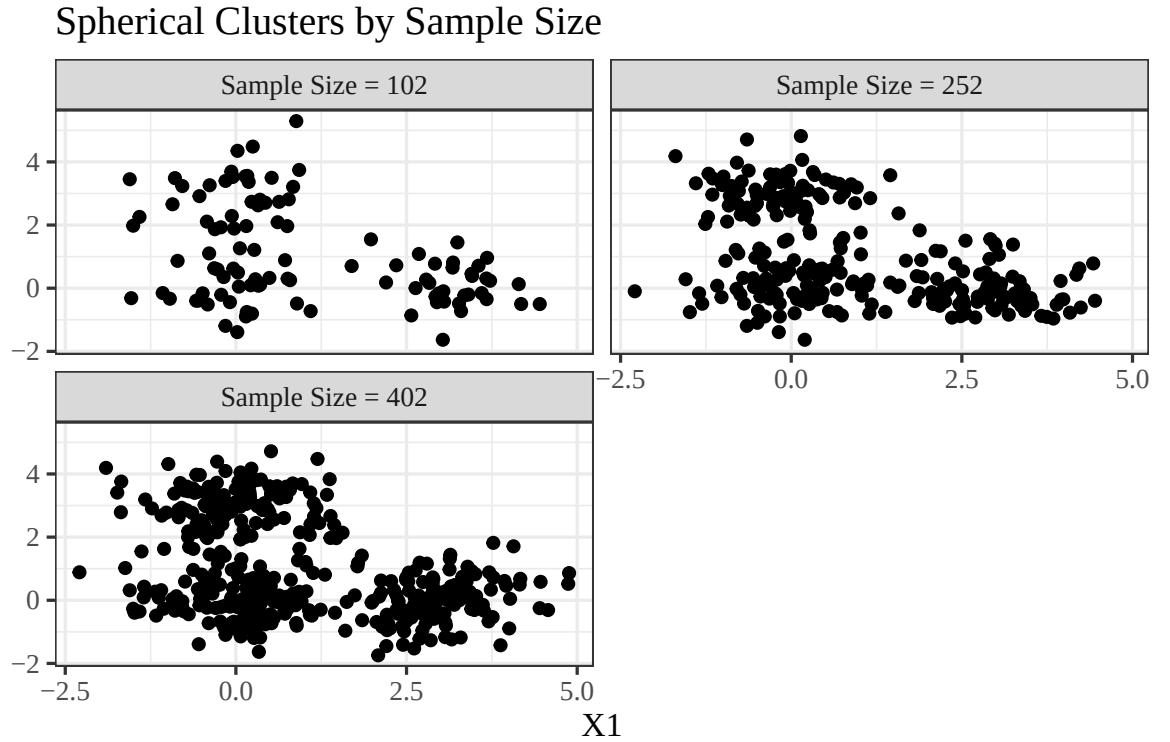


Figure 4.1: Sample Size Datasets

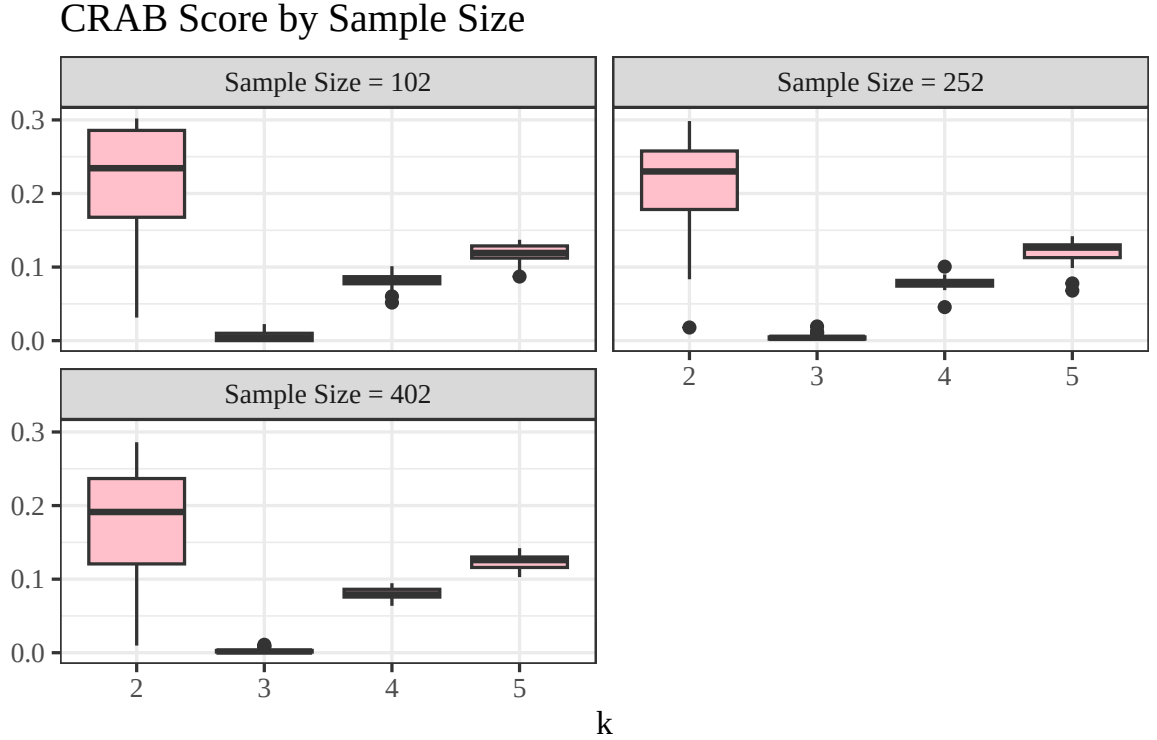


Figure 4.2: CRAB Score Across Sample Size

Figure 4.1 illustrates that as the sample size increases, the clusters become more densely packed, making the groupings easier to discern visually.

Despite these differences in sample size, CRAB consistently identifies the correct number of clusters. In all cases, $k = 3$ achieves the lowest CRAB score (Figure 4.2), demonstrating the algorithm's robustness across varying dataset sizes.

4.3.2 The Effect of Variance (σ^2)

To examine how cluster overlap affects CRAB's performance, we generated spherical clusters using the MVN distribution across variances 0.2, 0.5, 0.8, and 1. With MVN distributions, smaller variances produce tightly packed, and well separated clusters, while larger variances create more dispersed clusters with significant overlap.

For each scenario, $k = 3$ remains the true cluster structure, and the total sample size is set

to 252 observations to ensure an equal number of points per cluster.

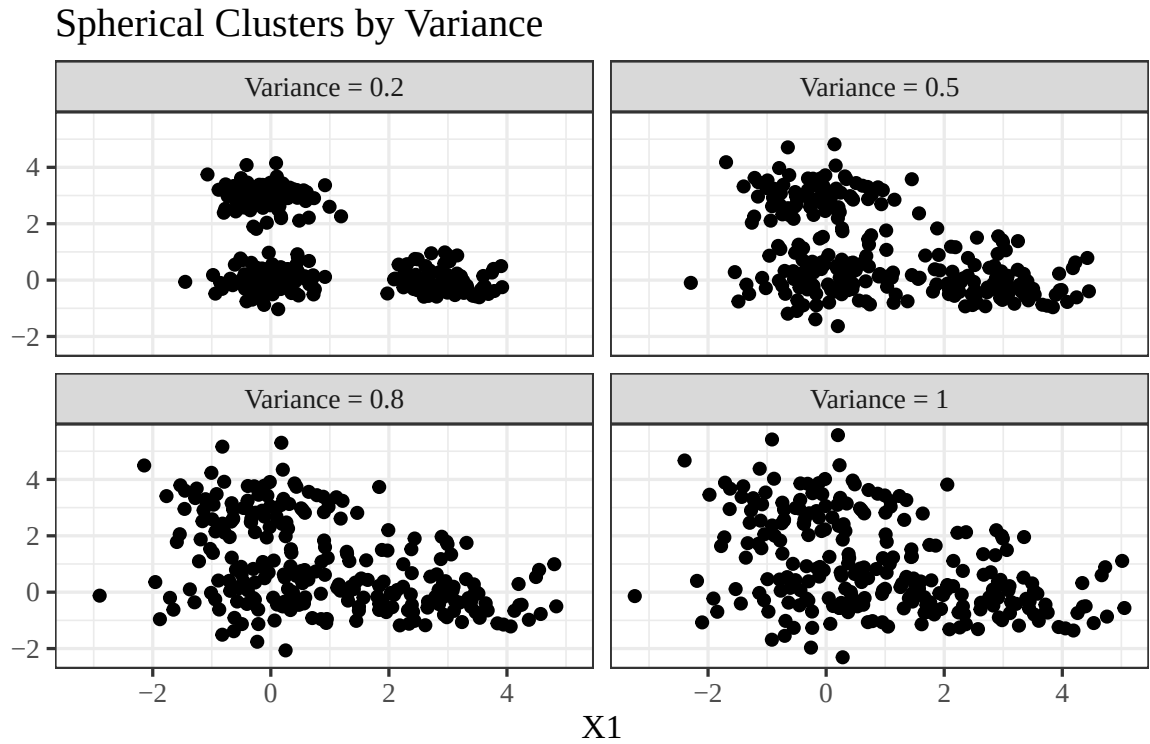


Figure 4.3: Spherical Clusters by Variance

Figure 4.3 presents four side by side scatterplots, each corresponding to a different variance (0.2, 0.5, 0.8, 1), illustrating how cluster spread affects visual separability. At lower variances, the clusters are clearly distinct and easily identifiable, but as variance increases, the boundaries become less defined, making it more challenging to discern the underlying structure by eye.

Multiple Seeds Testing CRAB Score Across Variance

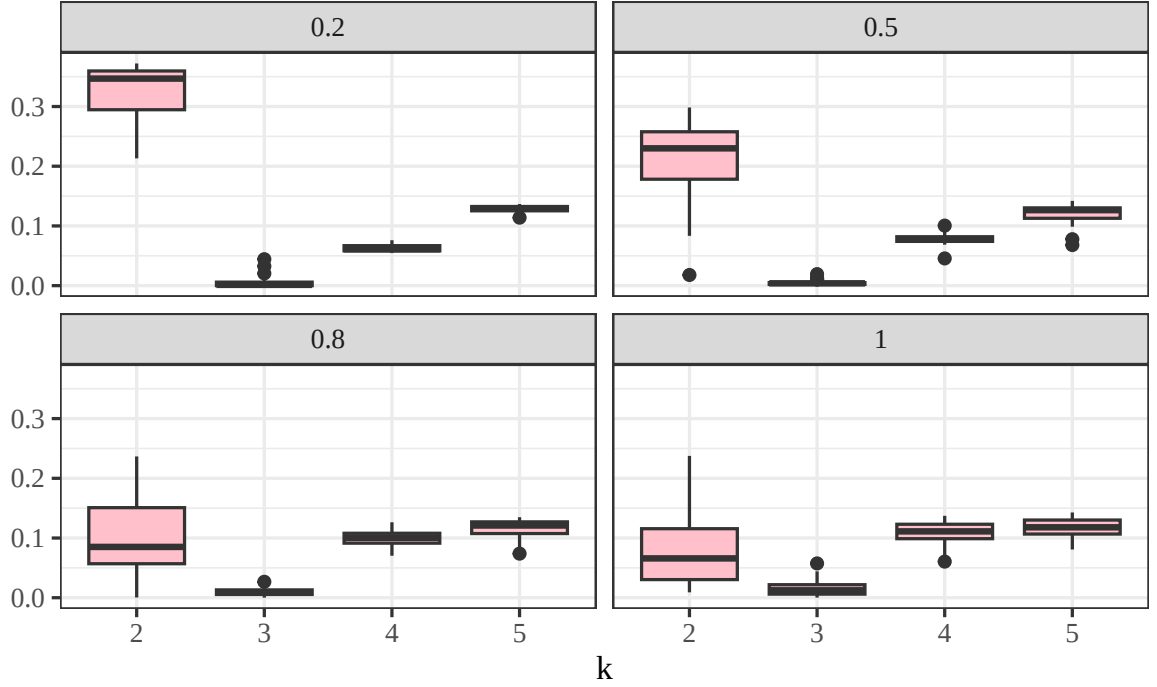


Figure 4.4: Multiple Seeds Testing CRAB Score Across Variance

Table 4.1: Win Proportion for Each k Across Different Variances

Variance	$k = 2$	$k = 3$	$k = 4$	$k = 5$
0.2	0%	100%	0%	0%
0.5	0%	100%	0%	0%
0.8	8%	92%	0%	0%
1.0	12%	88%	0%	0%

Note:

Proportions use 25 runs per variance.

Across 25 runs, CRAB correctly identified $k = 3$ in all runs for the lower variances of 0.2 and 0.5, achieving a perfect success rate. As the variance increased, the algorithm continued to identify the correct number of clusters in the majority of runs. At variance 0.8, CRAB

predicted $k = 3$ in 92 percent of runs and $k = 2$ in 8 percent. Similarly, while at variance 1, it predicted $k = 3$ in 88 percent of runs and $k = 2$ in 12 percent (Table 4.1).

These results demonstrate that CRAB reliably recovers the correct number of groupings under low to moderate overlap. Even as cluster separation decreases with higher variance, the algorithm remains largely robust, though prediction certainty declines slightly.

4.3.3 The Effect of Number of Clusters (k)

For this simulation, we evaluated whether the CRAB algorithm can correctly identify the true underlying structure when the correct number of groupings changes. While $k = 3$ provides a useful baseline, it is important that CRAB accurately captures the correct k across multiple scenarios. Here, we test CRAB's performance for $k = 2$, $k = 3$, and $k = 4$.

For each scenario, we set the variance to 0.5, and similar to the simulations in Section 4.3.2, we adjusted the total sample size to 252 observations are equally split among each of the tested cluster groupings.

Each test k value is represented by two types of plots:

1. **“Individual Seeds” Plot:** Three separate runs of the CRAB algorithm
2. **“Multiple Seeds” Plot:** Side by side boxplots summarizing results from 25 runs for each k value

The individual seeds plot reveals cases where a particular k value may emerge as optimal, which might be less apparent in the aggregated boxplots. The three individual runs correspond to the first three seeds used in the multiple seeds plot.

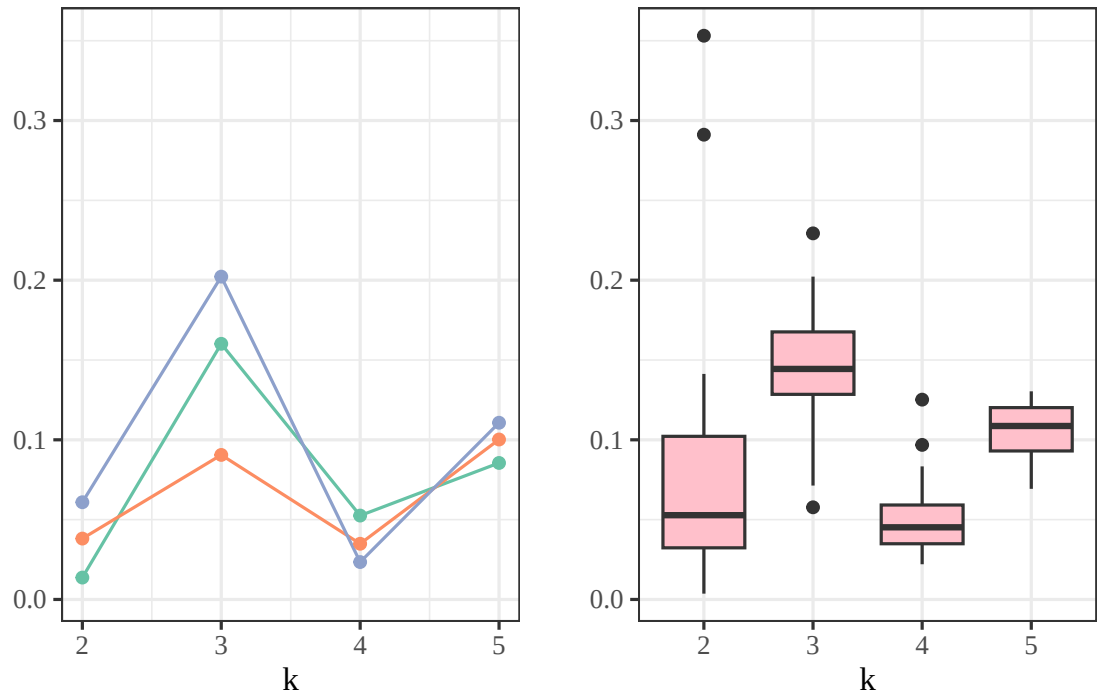


Figure 4.5: CRAB Scores for Individual and Multiple Seeds When $k = 2$

Table 4.2: Cluster k Win Proportion When $k = 2$

k	Number of Wins	Win Proportion
2	10	40%
3	0	0%
4	15	60%
5	0	0%

Note:

Proportions use 25 runs per variance.

Examining the individual runs for $k = 2$, two out of three seeds (purple and orange lines) indicate $k = 4$ as the optimal number of clusters (Figure 4.5). In particular, the purple line shows a strong preference for $k = 4$, while the orange line produces very similar CRAB

scores for $k = 2$ and $k = 4$. The green line, on the other hand, distinctly favors $k = 2$ over $k = 3, 4$, and 5 .

The multiple seeds plot in Figure 4.5 highlights that CRAB scores for $k = 2$ are more variable, whereas scores for $k = 4$ remain consistently close to zero. This higher variability for $k = 2$ reflects plausible alternative cluster structures rather than indicating an error, since clusters of two can naturally subdivide.

Across all 25 runs, the algorithm selects $k = 4$ most frequently, winning 60 percent of the time, while $k = 2$ accounts for the remaining 40 percent (Table 4.2). The algorithm never selects any of the other tested k values as the most optimal.

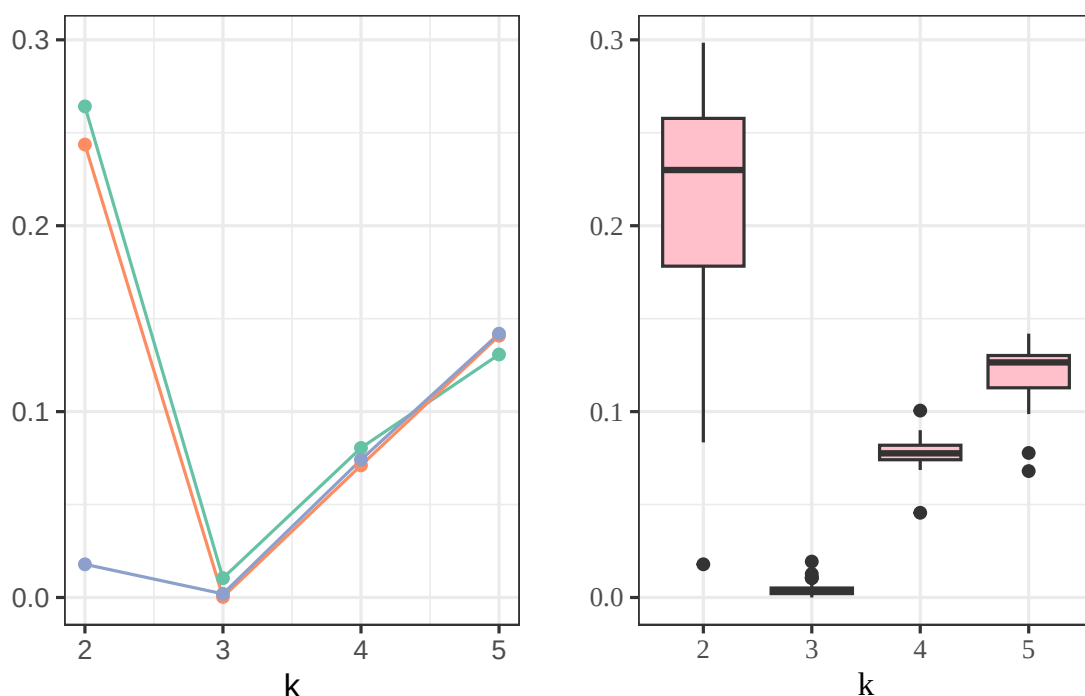


Figure 4.6: CRAB Scores for Individual and Multiple Seeds When $k = 3$

When $k = 3$, the individual seeds consistently point to three clusters as the winner using the global minimum of the CRAB scores (Figure 4.6). Across the 25 runs, CRAB scores for $k = 3$ show the smallest range and are the closest to zero compared to the other k

values, suggesting a strong preference for this number of clusters. In this case, the algorithm successfully identifies the underlying structure, and the result is reinforced by a clear pinching point, where the CRAB score dips sharply at $k = 3$ before rising for higher k values.

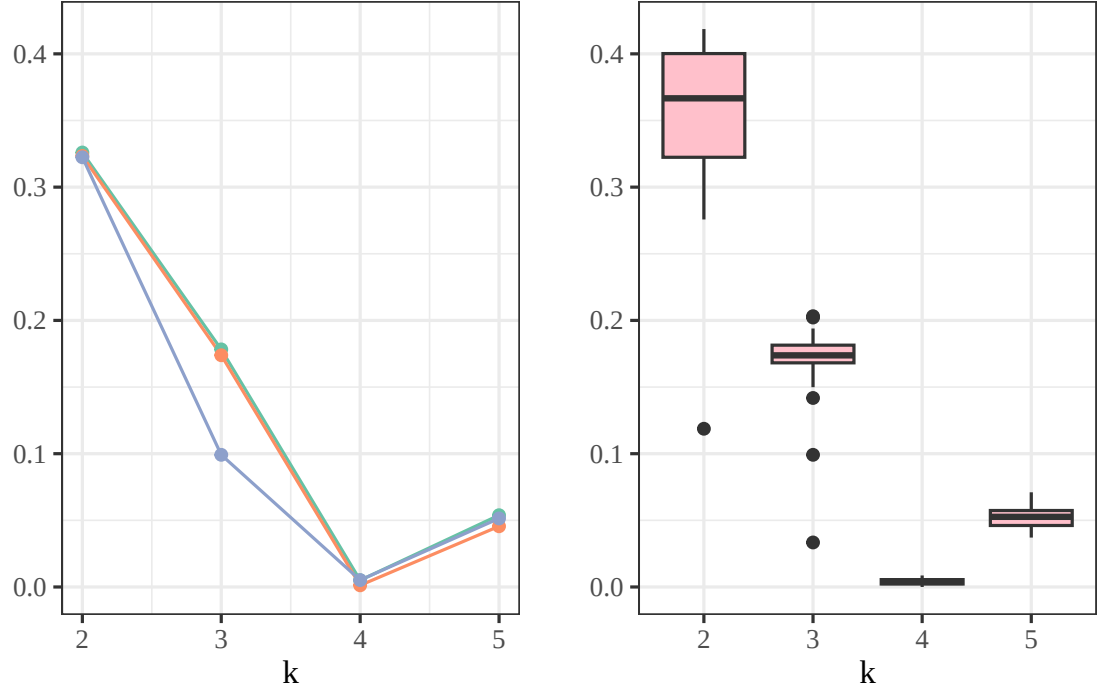


Figure 4.7: CRAB Scores for Individual and Multiple Seeds When $k = 4$

Testing $k = 4$ produces a similar pattern. The algorithm identifies four clusters as the best choice (Figure 4.7), with CRAB scores again concentrated near zero and a distinct pinching point at $k = 4$. Although the boxplots show that $k = 3$ and $k = 5$ exhibit relatively small ranges, neither reach the low scores observed for $k = 4$.

Overall, CRAB consistently identifies the correct number of clusters across different k values. For $k = 3$ and $k = 4$, the algorithm reliably selects the true cluster count, with CRAB scores concentrated near the global minimum and distinct pinching points. CRAB scores varied for $k = 2$, suggesting four clusters in the majority of runs, suggesting plausible alternative subdivisions of the data. These results confirm that CRAB accurately recovers the underlying clustering configuration and maintains stability across repeated runs.

Across all examined factors, including sample size, variance, and the number of clusters, CRAB demonstrates robust and reliable performance on spherical data. The algorithm consistently identifies the correct number of clusters under low to moderate overlap. Even as cluster separation decreases with higher variance or as larger sample sizes increase cluster density, its accuracy is maintained.

4.4 Non-Spherical Data

To evaluate CRAB's performance on non-spherical clusters, we generated elongated and irregular cluster shapes while varying sample size and the number of clusters. Variance was not systematically tested, as each cluster naturally exhibits its own spread in these non-spherical configurations. This approach allows us to examine how deviations from spherical geometry affect the algorithm's ability to identify cluster structure.

4.4.1 Effect of Sample Size

For non-spherical clusters, sample size can influence how well CRAB captures the underlying structure given the irregular and elongated shapes of the clusters. We examined datasets with 102, 252 and 402 observations, keeping the correct number of clusters fixed at 3. To ensure balanced comparisons, we distributed the observations evenly across clusters in each scenario, providing a consistent basis for evaluating CRAB's performance.

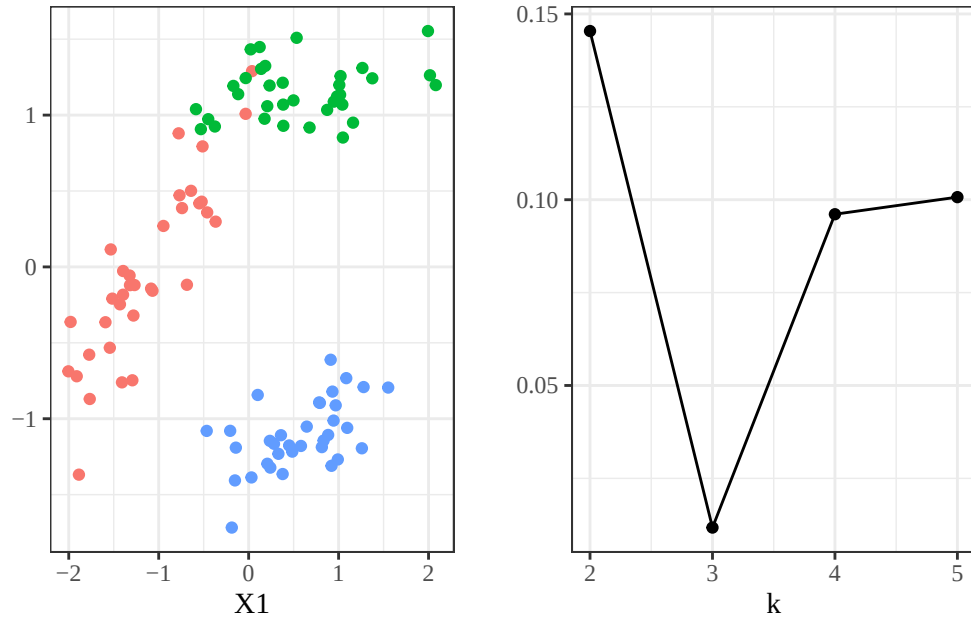


Figure 4.8: CRAB Scores for Non-Spherical Data with Sample Size = 102

With a sample size of 102, each of the three clusters contains only 34 observations. Despite the relatively small number of points per cluster, the CRAB algorithm successfully identified the correct number of clusters (Figure 4.8). The resulting CRAB score plot exhibits a clear pinching point at the true solution, confirming that $k = 3$ represents the optimal clustering configuration.

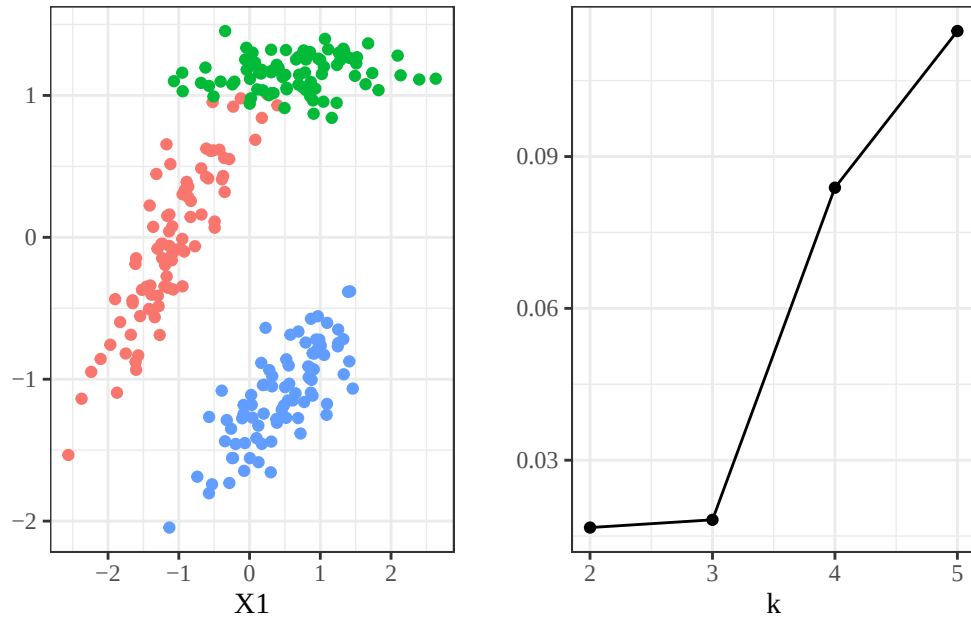


Figure 4.9: CRAB Scores for Non-Spherical Data with Sample Size = 252

Increasing the total sample size to 252, however, alters the outcome. In this scenario, CRAB identifies $k = 2$ as the best solution, narrowly outperforming $k = 3$ in terms of the lowest score (Figure 4.9).

To investigate this behavior, we applied the `point_contribution()` function. Points highlighted in bright red indicate the largest contributions to the CRAB score and, therefore, have the greatest influence on the algorithm's evaluation.

The point contribution scatterplot for $k = 2$ (Figure 4.10) reveals several high contribution points near $(-1, 0.5)$ to (0.05) and around $(1.5, -0.8)$. These points align with the boundaries determined by K-means when clustering the data into two centroids, suggesting that the assignment of points by K-means is a driving factor.

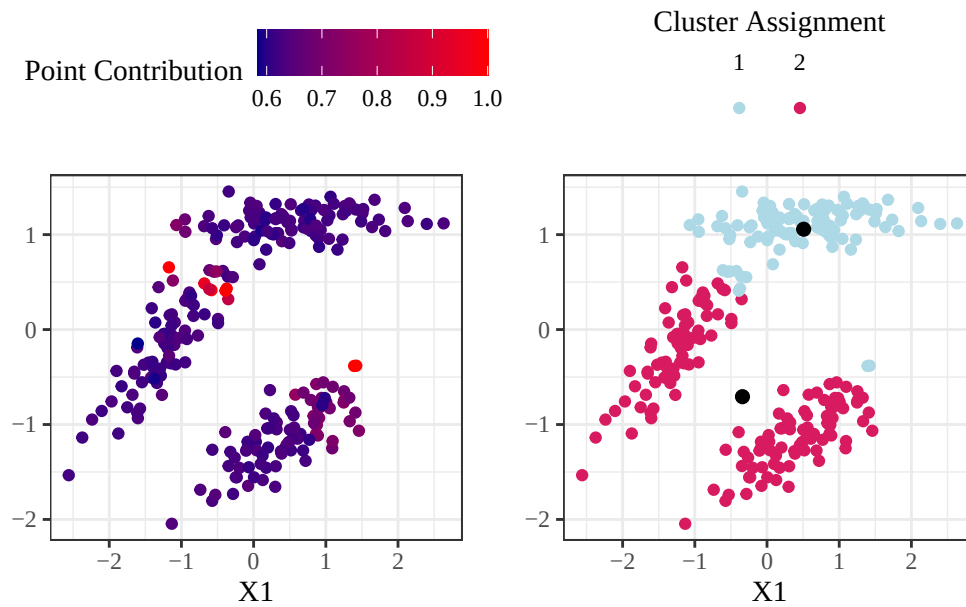


Figure 4.10: Point Contribution Scores for Sample Size = 252 with $k = 2$

A similar pattern emerges in the $k = 3$ scenario (Figure 4.11). While the bottom right cluster contains points with minimal contribution to the CRAB score, the top two clusters include slightly higher contributing points, with the brightest points matching the K-means cluster boundaries.

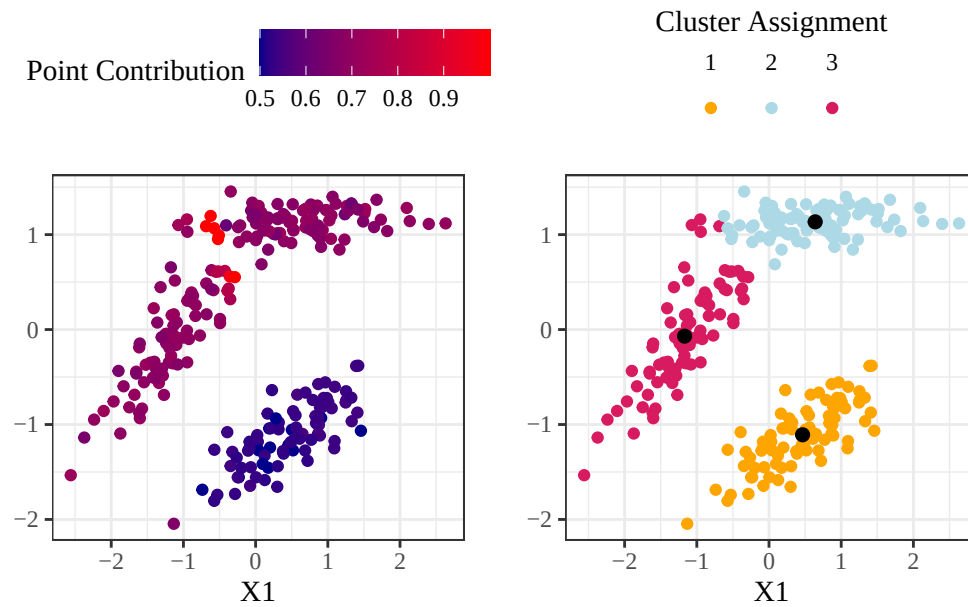


Figure 4.11: Point Contribution Scores for Sample Size = 252 with $k = 3$

These observations indicate that the deviation in CRAB's selection may not reflect a limitation of the algorithm itself but rather the influence of the K-means cluster assignments used in the implementation.

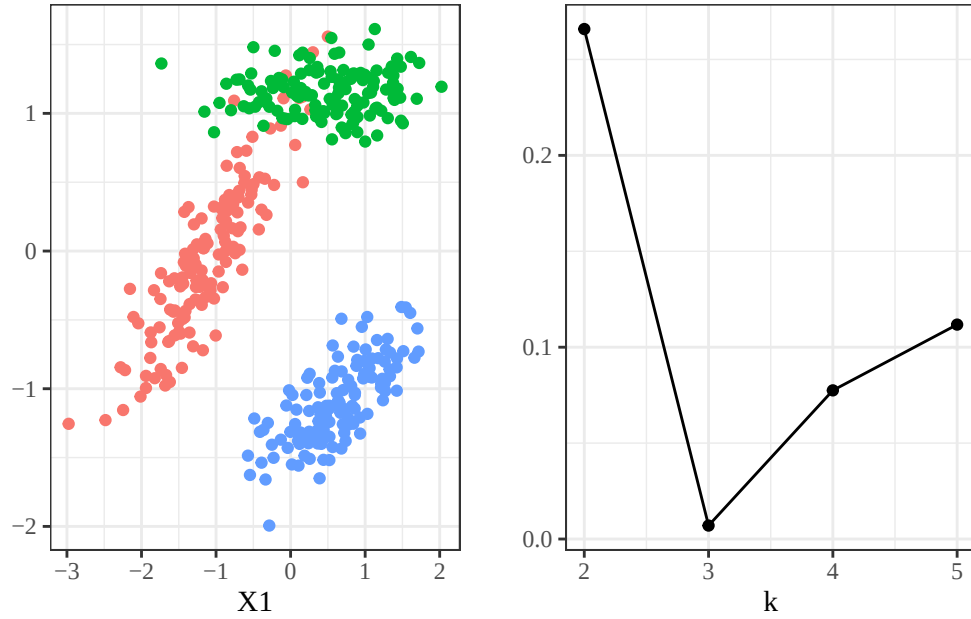


Figure 4.12: CRAB Scores for Non-Spherical Data with Sample Size = 402

When the sample size is increased to 402, the CRAB algorithm successfully detects the correct number of groupings. The CRAB score plot displays a pinching point at $k = 3$, illustrating that the algorithm can recover the true cluster structure when more observations are available, even for irregular and elongated cluster shapes.

Overall, for non-spherical clusters, CRAB reliably determines the true number of clusters when the sample size is small (102 observations) or large (402 observations), as indicated by clear pinching points and global minimums at $k = 3$. Intermediate sample sizes can lead to temporary deviations, where high contribution points influence the score and slightly shift the optimal k , potentially due to K-means cluster assignments. Increasing the number of observations enhances CRAB's stability and accuracy in capturing irregular cluster structures.

4.4.2 Effect of Cluster

To evaluate the effect of the number of clusters on non-spherical data, we placed the initial clusters in consistent locations, adding additional clusters incrementally when called for (Figure 4.13). For example, the $k = 2$ example contains two clusters. In the $k = 3$ scenario, the two clusters that were present in the $k = 2$ simulation are kept and a third cluster is introduced. The original two clusters are slightly adjusted to maintain a fixed total of 252 observations. Since we could not vary cluster placement or shape extensively, we increased the number of resamples to 60 and set the sample proportion to 0.90 to ensure a more robust assessment. Unlike the spherical simulations, we did not fix variance, allowing each elongated cluster to retain its natural variability, which provides a more realistic and challenging test for CRAB.

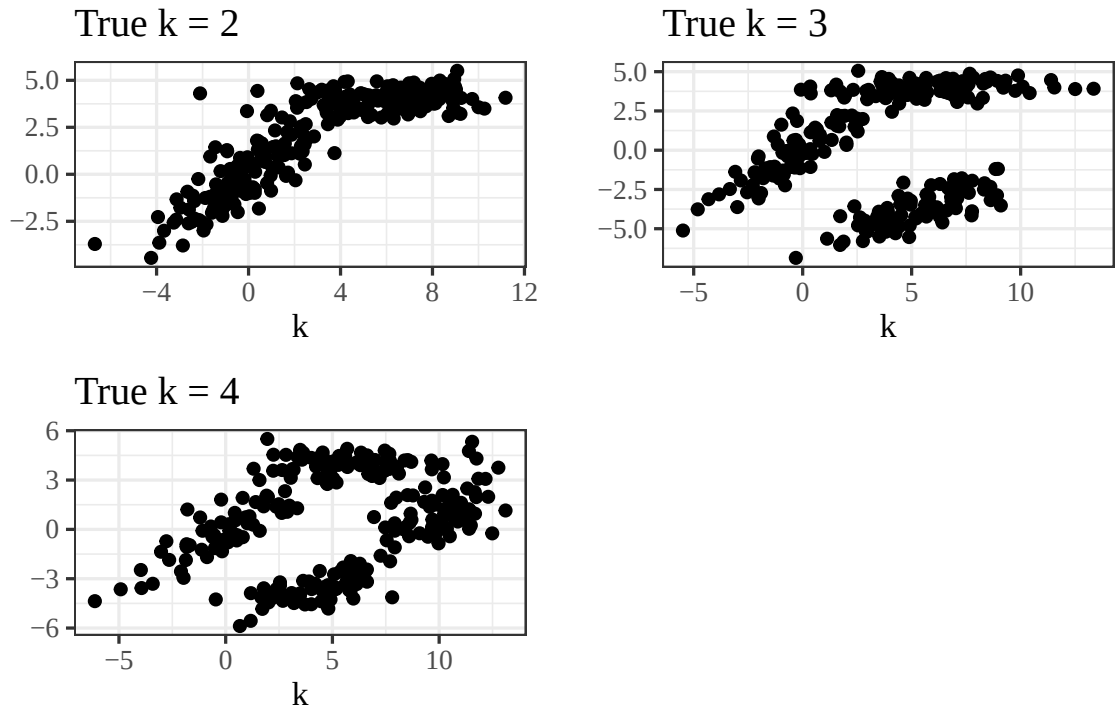


Figure 4.13: Non-Spherical Data With Different k Values

Figure 4.14 displays the outcome of the $k = 2$ scenario. Here, CRAB identified two clusters

as the optimal solution. However, the score for $k = 4$ followed closely behind, reflecting a similar pattern observed in Section 4.4.1, where datasets with two true clusters sometimes produced competing minima at $k = 2$ and $k = 4$.

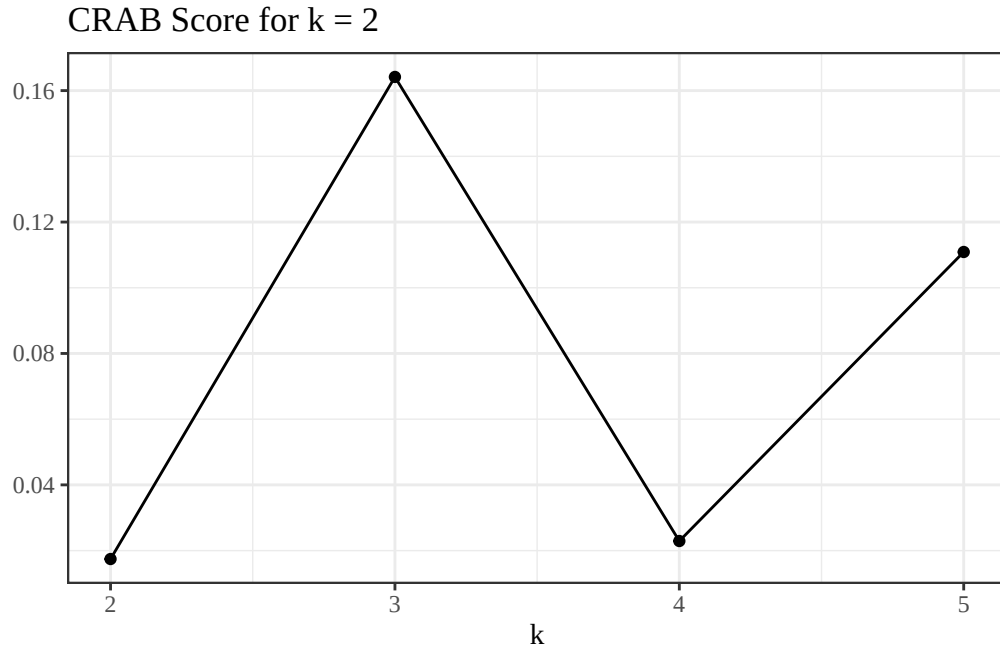


Figure 4.14: CRAB Scores for Non-Spherical Data with $k = 2$

When the number of clusters increased to $k = 3$, CRAB successfully detected the correct three cluster structure, producing a subtle but discernible pinching point (Figure 4.15).

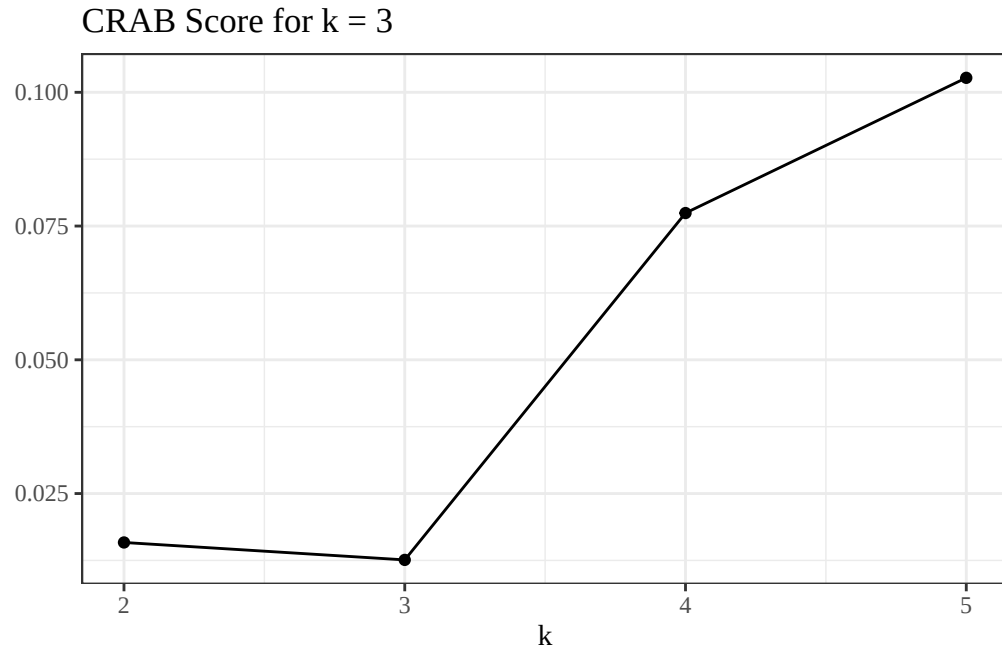


Figure 4.15: CRAB Scores for Non-Spherical Data with $k = 3$

Similarly, in the $k = 4$ scenario, the algorithm accurately identified four distinct clusters, reinforcing CRAB's ability to capture the true cluster structure as complexity and irregularity grow.

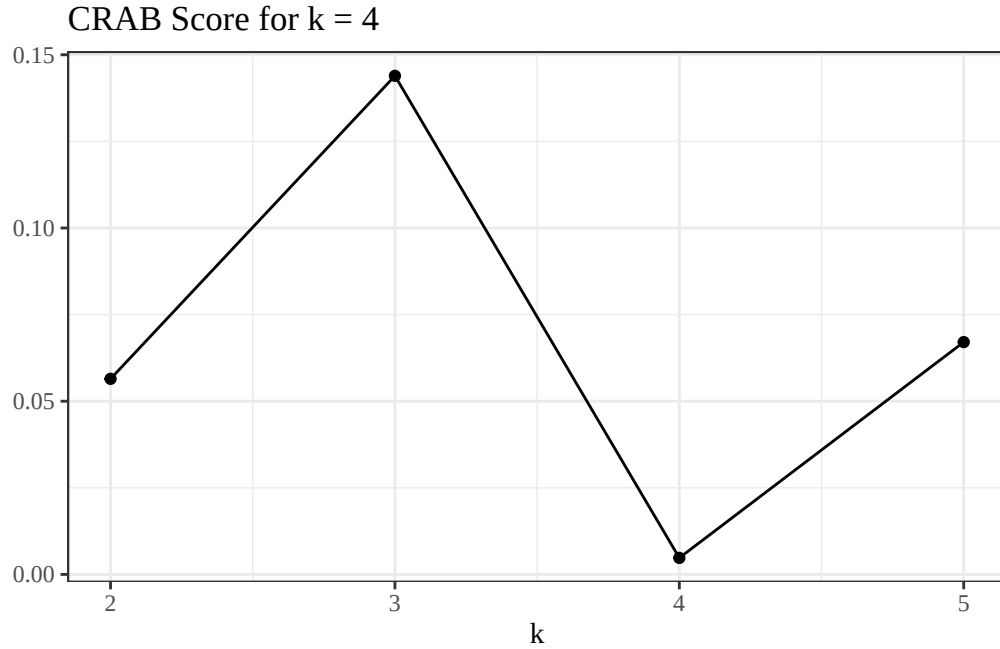


Figure 4.16: CRAB Scores for Non-Spherical Data with $k = 4$

Across the different cluster scenarios, CRAB consistently identified the true number of groupings, even as additional clusters were introduced. While minor ambiguity appeared in the $k = 2$ scenario, the algorithm reliably detected $k = 3$ and $k = 4$ structures. These results demonstrate that CRAB can adapt to increasing cluster complexity and maintain accuracy.

Overall, the simulations show that CRAB performs robustly on non-spherical data, even when clusters are elongated, variances differ and sample sizes vary. The algorithm found the correct number of clusters for larger sample sizes and higher cluster complexity, with the point contribution analysis highlighting how individual observations influence score outcomes. In combination with the spherical data simulations, these results provide strong evidence that CRAB can reliably detect underlying structures.

Chapter 5

CONCLUSION

Unsupervised learning and clustering present a fundamental challenge because the absence of a true target variable makes it difficult to determine the correct number of clusters. Common evaluation methods, such as the Elbow Method and the Silhouette Score can sometimes produce ambiguous results and often rely on assumptions about cluster shape or separation. While these methods are helpful, there is room for improvement. Stability offers an alternative approach by assessing clustering quality across multiple subsamples of a dataset. When points consistently group together across subsamples, it indicates a reliable cluster structure. The CRAB algorithm applies this principle by prioritizing clusters where points in the same cluster stay together. To make the method accessible, we developed an R package that emphasizes a tidy and efficient workflow and provides an intuitive syntax.

With this package, we tested the CRAB algorithm on both spherical and non-spherical datasets. For spherical data, we examined the effects of sample size, variance, and the true number of groupings. Across all sample sizes, CRAB consistently recovered the underlying structure. For lower variances, it produced the correct arrangement every time, while higher variances reduced its accuracy slightly. When varying the number of groups, the algorithm struggled with two clusters, often classifying them as four, but consistently identified three and four clusters correctly. For non-spherical data, we evaluated sample size and the number of groupings while preserving the individual variances of the elongated clusters. CRAB successfully detected three groups for the smallest and largest sample sizes. However, two clusters slightly outperformed three in the medium sample, potentially due to the K-means cluster assignments. Across tests with different group numbers, the algorithm showed some difficulty with two clusters, but reliably was able to distinguish two, three and four clusters.

Overall, CRAB effectively captures stable structures across diverse datasets, even when shapes and variances vary.

Despite these strengths, the CRAB algorithm has a few limitations. Its computational complexity grows quadratically with sample size, which can slow performance on larger datasets. The current evaluation also focuses exclusively on K-means clustering which may limit its generalizability. Future work could apply CRAB to a wider range of datasets and clustering algorithms to test its robustness. It would also be valuable to conduct simulation studies to examine how parameters such as subsample proportion and the number of subsamples affect the CRAB score. Further simulations could explore cluster numbers like three, four, six and eight to determine whether smaller groupings might naturally subdivide and be mistaken for more clusters.

In conclusion, CRAB effectively uncovers stable cluster structures and lays a new foundation for future applications in clustering. By emphasizing consistency across subsamples, the algorithm provides a reliable approach for determining the optimal number of clusters in datasets where traditional methods may be unclear or unreliable.

REFERENCES

- Bache, S. M., & Wickham, H. (2025). *Magrittr: A forward-pipe operator for r*. <https://doi.org/10.32614/CRAN.package.magrittr>
- Ben-Hur, A., Elisseeff, A., & Guyon, I. (2002). A stability based method for discovering structure in clustered data. *Pacific Symposium on Biocomputing. Pacific Symposium on Biocomputing, 2002*, 6–17. https://doi.org/10.1142/9789812799623_0002
- Corporation, M., & Weston, S. (2022). *doParallel: Foreach parallel adaptor for the 'parallel' package*. <https://doi.org/10.32614/CRAN.package.doParallel>
- Horst, A. M., Hill, A. P., & Gorman, K. B. (2020). *Palmerpenguins: Palmer archipelago (antarctica) penguin data*. <https://doi.org/10.5281/zenodo.3960218>
- Hvitfeldt, E., & Bodwin, K. (2025). *Tidyclust: A common API to clustering*. <https://doi.org/10.32614/CRAN.package.tidyclust>
- Jaeger, A., & Banks, D. (2023). Cluster analysis: A modern statistical review. *WIREs Computational Statistics*, 15(3), e1597. <https://doi.org/https://doi.org/10.1002/wics.1597>
- Kodinariya, T., & Makwana, P. (2013). Review on determining of cluster in k-means clustering. *International Journal of Advance Research in Computer Science and Management Studies*, 1, 90–95.
- Kuhn, M., & Wickham, H. (2020). *Tidymodels: A collection of packages for modeling and machine learning using tidyverse principles*. Retrieved from <https://www.tidymodels.org>
- Microsoft, & Weston, S. (2022). *Foreach: Provides foreach looping construct*. <https://doi.org/10.32614/CRAN.package.foreach>
- R Core Team. (2026). *R: A language and environment for statistical computing*. Vienna, Austria: R Foundation for Statistical Computing. Retrieved from <https://www.R-project.org/>
- Venables, W. N., & Ripley, B. D. (2002). *Modern applied statistics with s* (Fourth). New

York: Springer. Retrieved from <https://www.stats.ox.ac.uk/pub/MASS4/>

APPENDICES

Appendix A

BUDDIES_MATRIX()

```
buddies_matrix <- function(data = data,
                             k = 3,
                             subsample_prop = 0.8,
                             num_subsamples = 10,
                             start_seed = 123,
                             cluster_method){

  ### INPUT CHECKS:

  # data is a dataframe
  if (!is.data.frame(data)) {
    stop("Data must be in a dataframe.")}

  # k is a positive int
  if (k %% 1 != 0 || k < 1) {
    stop("The desired k value must be a positive integer.")}

  # k is 1
  if (k == 1) {
    warning("The desired k value is 1 (one cluster).")}
```

```

# sub_sample prop is between 0 and 1
if (subsample_prop <= 0 || subsample_prop >= 1) {
  stop("Subsample proportion needs to be between 0 and 1.")}

# num_subsamples is a positive int
if (num_subsamples <= 0 || num_subsamples %% 1 != 0) {
  stop("The number of subsamples must be a positive integer.")}

# clustering method is kmeans
if (cluster_method != "kmeans"){
  stop("Error: Clustering method does not currently work.
       Please change to 'kmeans'.")}]

# start seed is a positive int
if (start_seed %% 1 != 0 || start_seed < 0) {
  stop("The seed value must be an integer.")}

### FUNCTION:
# parallelization
cores <- max(1, parallel::detectCores() - 5)
cl <- parallel::makeCluster(cores)
doParallel::registerDoParallel(cl)
on.exit(parallel::stopCluster(cl))

# drop na and scale data
data_clean <- data |>
  tidyr::drop_na() |>

```

```

scale() |>
as.data.frame() |>
dplyr::mutate(index = dplyr::row_number())

# initialize results list -- holds matrix for a given k
results <- list()
results <- foreach::`%dopar%`(
  foreach::foreach(i = 1:num_subsamples,
                    .packages = c("tidyclust",
                                   "tidyverse",
                                   "tidymodels")), {

# initializing blank matrix
result_matrix <- matrix(0, nrow = nrow(data_clean),
                        ncol = nrow(data_clean))

# seed
set.seed(start_seed + i)

# resample
random_sample <- data_clean |>
  filter(index %in%
          sample(index, floor(subsample_prop * nrow(data))))

# clustering
if (cluster_method == "kmeans"){
  model <- tidyclust::k_means(num_clusters = k) |>

```

```

tidyclust::fit(~ .,
  data = random_sample[,
    !(names(random_sample) %in% "index")]]}

else{stop("Model option currently in progress.")}

# assign points to cluster
intermediate <- data.frame(
  random_sample$index,
  tidyclust::extract_cluster_assignment(model) |>
    mutate(.cluster = as.character(.cluster)),
  stringsAsFactors = FALSE)

# rename columns
colnames(intermediate) <- c("index", "cluster")

# assign results matrix (-1, 1, 0)
for (c in unique(intermediate$cluster)){
  idx <- intermediate[intermediate$cluster == c, ]$index

  if (length(idx) > 1){
    idx <- sort(unlist(idx), method = "radix")
    ones <- t(combn(idx, 2))
    result_matrix[ones[, 1], ones[, 2]] <- 1}

  neg_one_idx <- expand.grid(idx,

```

```

        setdiff(random_sample$index, idx))

result_matrix[neg_one_idx[, 1],
              neg_one_idx[, 2]] <- -1}

# set lower triangle and diagonal to NA
result_matrix[lower.tri(result_matrix, diag = TRUE)] <- NA

        return(result_matrix)})

# return results
return(results)}

```

Appendix B

CRAB()

```
crab <- function(data,
                 n = 100,
                 min_k = 2,
                 max_k = 5,
                 subsample_prop = 0.8,
                 num_subsamples = 10,
                 start_seed = 123,
                 cluster_method){

  ### INPUT CHECKS:

  # data is a dataframe
  if (!is.data.frame(data)) {
    stop("Data must be in a dataframe.")}

  # min_k is a positive int
  if (min_k %% 1 != 0 || min_k < 1) {
    stop("The minimum k value must be a positive integer.")}

  # min_k is 1
  if (min_k == 1) {
    warning("The minimum k value is 1 (one cluster).")}
```

```

# min_k < max_k
if (min_k >= max_k) {
  stop("The minimum k value must be less than the maximum
      k value.")}

# max_k is a positive int
if (max_k %% 1 != 0 || max_k < 1) {
  stop("The maximum k value must be a positive integer.")}

# max_k is 1
if (max_k == 1) {
  warning("The maximum k value is 1 (one cluster).")}

# sub_sample prop is between 0 and 1
if (subsample_prop <= 0 || subsample_prop >= 1) {
  stop("Subsample proportion needs to be between 0 and 1.")}

# num_subsamples is a positive int
if (num_subsamples <= 0 || num_subsamples %% 1 != 0) {
  stop("The number of subsamples must be a positive integer.")}

# clustering method is kmeans
if (cluster_method != "kmeans"){
  stop("Error: Clustering method does not currently work.
      Please change to 'kmeans'.")}

# start seed is a positive int

```

```

if (start_seed %% 1 != 0 || start_seed < 0) {
  stop("The seed value must be an integer.")}

### FUNCTION:

# initialize score list
k_scores <- data.frame(k = numeric(),
                      Score = numeric(),
                      `Time (seconds)` = numeric(),
                      `Best Cluster (k)` = character(),
                      check.names = FALSE)

# iterate over k values
for (k in min_k:max_k){

  # start time
  tictoc::tic()

  # get buddies matrices and overall mean matrix
  mean_mat <- mean_matrix(data = data,
                        k = k,
                        subsample_prop = subsample_prop,
                        num_subsamples = num_subsamples,
                        start_seed = start_seed,
                        cluster_method = cluster_method)

  # store all scores
  score <- similarity_score(mean_matrix = mean_mat)

```



```

# end time

time_info <- tictoc::toc(quiet = TRUE)

# calculate execution time

elapsed_time <- time_info$toc - time_info$tic

# add to results

k_scores <- rbind(k_scores,
                  data.frame(k = k,
                              Score = score,
                              `Time (seconds)` = elapsed_time,
                              `Best Cluster (k)` = " ",
                              check.names = FALSE,
                              stringsAsFactors = FALSE))}

# find row with lowest score

best_score_idx <- which.min(k_scores$Score)

k_scores$`Best Cluster (k)`[best_score_idx] <- "BEST"

# remove row names index

rownames(k_scores) <- NULL

# get similarity score

return(k_scores)}

```

Appendix C

CRAB_BAKE()

```
crab_bake <- function(data_info){

  ### INPUT CHECKS:

  # check if data_info is a list
  if (!is.list(data_info)) {
    stop("Input must be a list (output from crab_prep function).")
  }

  # correct element names
  required_names <- c("data", "original_cols",
                      "specified_cols", "specified_params")
  if (!all(required_names %in% names(data_info))) {
    stop("Input list is missing at least one of the following:
          'data', 'original_cols', 'specified_cols',
          'specified_params.' Did you provide the correct object?")
  }

  # check data is a dataframe
  if (!is.data.frame(data_info$data)) {
    stop("The 'data' element must be a dataframe.")
  }

  # check original_cols is a list
  if (!is.character(data_info$original_cols)) {
    stop("original_cols must be a character vector of column names.")
  }
}
```

```

# check specified_params is a list
if (!is.list(data_info$specified_params)) {
  stop("The 'specified_params' element must be a list.")}

# check cols are chosen
if (is.null(data_info$specified_cols)){
  stop("No columns have been specified.
       Call crab_cols() first.")}

# check params are chosen
if (is.null(data_info$specified_params)){
  stop("No parameters have been specified.
       Call crab_params() first.")}

### FUNCTION:

# grab data with chosen cols
crab_data <- data_info$data[,
                           data_info$specified_cols, drop = FALSE]

# grab params
params <- data_info$specified_params

# call crab()
score <- crab(data = crab_data,
              min_k = params$min_k,
              max_k = params$max_k,

```

```
        subsample_prop = params[subsample_prop],  
        num_subsamples = params[num_subsamples],  
        cluster_method = params[cluster_method])  
  
    return(score)}
```

Appendix D

CRAB_COLS()

```
crab_cols <- function(data_info, chosen_cols){  
  ### INPUT CHECKS:  
  # check if data_info is a list  
  if (!is.list(data_info)) {  
    stop("Input must be a list (output from crab_prep function).")  
  }  
  
  # correct element names  
  required_names <- c("data", "original_cols",  
                      "specified_cols", "specified_params")  
  if (!all(required_names %in% names(data_info))) {  
    stop("Input list is missing at least one of the following:  
        'data', 'original_cols', 'specified_cols',  
        'specified_params.' Did you provide the correct object?")  
  }  
  
  # check data is a dataframe  
  if (!is.data.frame(data_info$data)) {  
    stop("The 'data' element must be a dataframe.")  
  }  
  
  # check original_cols is a list  
  if (!is.character(data_info$original_cols)) {  
    stop("original_cols must be a character vector of column names.")  
  }  
}
```

```

# check chosen_cols is a character
if (!is.character(chosen_cols)) {
  stop("chosen_cols must be a character vector of column names.")}

### FUNCTION:

# check if chosen columns exist
missing_cols <- setdiff(chosen_cols, data_info$original_cols)

# leave error message if columns dont exist
if (length(missing_cols) > 0) {
  stop("Column(s) do not exist: ",
       paste(missing_cols, collapse = ", ")))}

# update specified cols
data_info$specified_cols <- chosen_cols

return(data_info)}

```

Appendix E

CRAB_PARAMS()

```
crab_params <- function(data_info, min_k, max_k,
                        num_subsamples, subsample_prop = 0.8,
                        cluster_method, start_seed = 123){

  ### INPUT CHECKS:

  # check if data_info is a list
  if (!is.list(data_info)) {
    stop("Input must be a list (output from crab_prep function).")
  }

  # correct element names
  required_names <- c("data", "original_cols",
                     "specified_cols", "specified_params")
  if (!all(required_names %in% names(data_info))) {
    stop("Input list is missing at least one of the following:
          'data', 'original_cols', 'specified_cols',
          'specified_params.' Did you provide the correct object?")
  }

  # check data is a dataframe
  if (!is.data.frame(data_info$data)) {
    stop("The 'data' element must be a dataframe.")
  }

  # check original_cols is a list
```

```

if (!is.character(data_info$original_cols)) {
  stop("original_cols must be a character vector of column names.")}

# min_k is a positive int
if (min_k %% 1 != 0 || min_k < 1) {
  stop("The minimum k value must be a positive integer.")}

# min_k is 1
if (min_k == 1) {
  warning("The minimum k value is 1 (one cluster).")}

# min_k < max_k
if (min_k >= max_k) {
  stop("The minimum k value must be less than the maximum k value.")}

# max_k is a positive int
if (max_k %% 1 != 0 || max_k < 1) {
  stop("The maximum k value must be a positive integer.")}

# max_k is 1
if (max_k == 1) {
  warning("The maximum k value is 1 (one cluster).")}

# sub_sample prop is between 0 and 1
if (subsample_prop <= 0 || subsample_prop >= 1) {
  stop("Subsample proportion needs to be between 0 and 1.")}

```



```

# num_subsamples is a positive int
if (num_subsamples <= 0 || num_subsamples %% 1 != 0) {
  stop("The number of subsamples must be a positive integer.")}

# clustering method is kmeans
if (cluster_method != "kmeans"){
  stop("Error: Clustering method does not currently work.
      Please change to 'kmeans'.")}]

### FUNCTION:

# update specified params
data_info$specified_params <- list(
  min_k = min_k,
  max_k = max_k,
  num_subsamples = num_subsamples,
  subsample_prop = subsample_prop,
  cluster_method = cluster_method,
  start_seed = start_seed)

return(data_info)}

```

Appendix F

CRAB_PREP()

```
crab_prep <- function(data){  
  
  ### INPUT CHECKS:  
  if (!is.data.frame(data)) {  
    stop("Data must be in a dataframe.")}  
  
  ### FUNCTION:  
  # grab all columns, leave user choices NULL  
  data_info <- list(data = data,  
                    original_cols = colnames(data),  
                    specified_cols = NULL,  
                    specified_params = list())  
  
  return(data_info)}
```

Appendix G

CRAB_SIM_PARAMS()

```
crab_sim_params <- function(data_info, min_k, max_k,
                             num_subsamples, n = 100, variance,
                             desired_k = 3, subsample_prop = 0.8,
                             num_runs, cluster_method,
                             start_seed = 123){

  ### INPUT CHECKS:

  # check min_k and max_k
  if (min_k > max_k){
    stop("Error: min_k must be <= max_k.")}

  if (min_k < 0){
    stop("Error: min_k must be > 0.")}

  if (max_k < 0){
    stop("Error: max_k must be > 0.")}

  # check subsample prop
  if (subsample_prop >= 1){
    stop("Error: subsample_prop must be < 1.")}

  # check num_runs
  if (num_runs < 0){
```

```

    stop("Error: num_runs must be > 0.")}

# check cluster method
if (cluster_method != "kmeans"){
    stop("Error: Clustering method does not currently work.
        Please change to 'kmeans'.")}

### FUNCTION:

# update specified params
data_info$specified_params <- list(
    min_k = min_k,
    max_k = max_k,
    num_subsamples = num_subsamples,
    n = n,
    variance = variance,
    desired_k = desired_k,
    subsample_prop = subsample_prop,
    num_runs = num_runs,
    cluster_method = cluster_method,
    start_seed = start_seed)

return(data_info)}

```

Appendix H

GET_POINTVEC()

```
get_pointvec <- function(i, mean_matrix_score) {  
  ### INPUT CHECKS:  
  
  # check score is a matrix  
  if (!is.matrix(mean_matrix_score)) {  
    stop("The 'score' element must be a matrix.")}  
  
  # check i is an integer  
  if (i %% 1 != 0 || i < 1) {  
    stop("Point must be a positive integer.")}  
  
  ### FUNCTION:  
  
  # grab i row and col, without diagonal  
  row <- mean_matrix_score[i, -i]  
  col <- mean_matrix_score[-i, i]  
  
  # grab all abs vals of i, removing NAs  
  res_vec <- c(row, col)  
  res_vec <- abs(res_vec[!is.na(res_vec)])  
  
  return(res_vec)}  
}
```

Appendix I

MEAN_MATRIX()

```
mean_matrix <- function(data = data,
                        k = 3,
                        subsample_prop = 0.8,
                        num_subsamples = 10,
                        start_seed = 123,
                        cluster_method){

  ### INPUT CHECKS:

  # data is a dataframe
  if (!is.data.frame(data)) {
    stop("Data must be in a dataframe.")}

  # k is a positive int
  if (k %% 1 != 0 || k < 1) {
    stop("The desired k value must be a positive integer.")}

  # k is 1
  if (k == 1) {
    warning("The desired k value is 1 (one cluster).")}

  # sub_sample prop is between 0 and 1
  if (subsample_prop <= 0 || subsample_prop >= 1) {
```

```

    stop("Subsample proportion needs to be between 0 and 1.")}

# num_subsamples is a positive int
if (num_subsamples <= 0 || num_subsamples %% 1 != 0) {
  stop("The number of subsamples must be a positive integer.")}

# clustering method is kmeans
if (cluster_method != "kmeans"){
  stop("Error: Clustering method does not currently work.
       Please change to 'kmeans'.")}

# start seed is a positive int
if (start_seed %% 1 != 0 || start_seed < 0) {
  stop("The seed value must be an integer.")}

### FUNCTION:

# call buddies_matrix()
results <- buddies_matrix(data,
                           k = k,
                           subsample_prop = subsample_prop,
                           num_subsamples = num_subsamples,
                           start_seed = start_seed,
                           cluster_method = cluster_method)

# abs value the entire matrix for all matrices
absolute_sum <- lapply(results, abs)

```

```
# get sum of abs value in final matrix
absolute_final <- absolute_sum |>
  purrr::reduce(`+`)

# get sum of values in final matrix
numerator <- results |>
  purrr::reduce(`+`)

return(list(matrices = results,
            score = numerator/absolute_final))}
```


Appendix J

POINT_CONTRIBUTION()

```
point_contribution <- function(data, mean_matrix) {  
  ### INPUT CHECKS:  
  # check if list  
  if (!is.list(mean_matrix)) {  
    stop("Input must be a list (output from mean_matrix function).")  
  }  
  
  # correct element names  
  required_names <- c("matrices", "score")  
  if (!all(required_names %in% names(mean_matrix))) {  
    stop("Input list is missing 'matrices' or 'score'.  
        Did you provide the correct object?")  
  }  
  
  # check score is a matrix  
  if (!is.matrix(mean_matrix$score)) {  
    stop("The 'score' element must be a matrix.")  
  }  
  
  # data is a dataframe  
  if (!is.data.frame(data)) {  
    stop("Data must be in a dataframe.")  
  }  
  
  ### FUNCTION:  
  # loop through every point
```

```

scores <- lapply(c(1:nrow(data)), function(i) {

  # abs vals of i
  full_vec <- get_pointvec(i, mean_matrix$score)

  # computer score
  singlePointScore <- mean((1 - full_vec)^2)
  return(list(index = i,
              singlePointScore = singlePointScore)))})

# create data frame
final_df <- do.call(rbind.data.frame, scores)

# mean score
total_score <- mean(final_df$singlePointScore)

# sigmoid fn
sigmoid <- function(x) 1 / (1 + exp(-x))

# point contribution
final_df <- final_df |>
  mutate(singlePointContribution =
          sigmoid(singlePointScore / total_score))

return(final_df)}

```

Appendix K

REPEAT_SIMULATIONS()

```
repeat_simulations <- function(crab_setup = NULL,
                               num_runs,
                               n = 100,
                               variance,
                               desired_k = 3,
                               min_k,
                               max_k,
                               subsample_prop = 0.8,
                               num_subsamples,
                               start_seed = 123,
                               cluster_method) {

  ### INPUT CHECKS:

  # setup object overwrites arguments if provided
  if (!is.null(crab_setup)){
    if (!is.list(crab_setup) ||
        is.null(crab_setup$specified_params)) {
      stop("crab_setup must be created using the pipeline:
           crab_prep() |> crab_sim_params().")
    }

    # unpack params
    params <- crab_setup$specified_params
```

```

num_runs      <- params$num_runs
n             <- params$n
min_k         <- params$min_k
max_k         <- params$max_k
subsample_prop <- params$subsample_prop
num_subsamples <- params$num_subsamples
cluster_method <- params$cluster_method
variance      <- params$variance
desired_k     <- params$desired_k

# will be null

input_data    <- crab_setup$data}

# no setup (also will be null)
else {input_data <- NULL}

# check variance
if (any(variance <= 0)){
  stop("All variance values must be greater than 0.")}

# check min_k
if (min_k %% 1 != 0 || min_k < 1) {
  stop("min_k must be a positive integer.")}

# check max_k
if (max_k %% 1 != 0 || max_k < 1) {
  stop("max_k must be a positive integer.")}

```

```

# check min_k < max_k
if (min_k >= max_k) {
  stop("min_k must be less than max_k.")}

# min_k = 1
if (min_k == 1 || max_k == 1) {
  warning("At least one of the min_k and max_k values
    provided is 1 (one cluster).")}

# num_runs
if (num_runs %% 1 != 0 || num_runs < 1) {
  stop("num_runs must be a positive integer.")}

# subsample_prop
if (subsample_prop <= 0 || subsample_prop >= 1) {
  stop("subsample_prop must be between 0 and 1.")}

# num_subsamples
if (num_subsamples %% 1 != 0 || num_subsamples <= 0) {
  stop("num_subsamples must be a positive integer.")}

if (num_subsamples < 5) {
  warning("A low number of subsamples may lead to
    unstable CRAB scores.")}

### FUNCTION:

```

```

# initialize results list
results_list <- list()
counter <- 1

# set seed so it starts at the same number specified
start_seed <- start_seed - 1

# loop through variances
for (var in variance) {

  # loop through number of runs/rep
  for (i in seq_len(num_runs)) {

    # alternate seed
    current_seed <- start_seed + i

    # simulate data
    sim_data <- simulate_mvn(n = n,
                             variance = var,
                             desired_k = desired_k,
                             start_seed = current_seed)

    # call crab()
    sim_result <- crab(data = sim_data,
                       start_seed = current_seed,
                       n = n,
                       min_k = min_k,

```

```

        max_k = max_k,
        subsample_prop = subsample_prop,
        num_subsamples = num_subsamples,
        cluster_method = cluster_method)

# add rep and variance identifier
sim_result$run <- i
sim_result$var <- var

# store result
results_list[[counter]] <- sim_result
counter <- counter + 1}}

# combine all results and get rid of index row names
all_results <- do.call(rbind, results_list)
rownames(all_results) <- NULL

return(all_results)}

```

Appendix L

SIMILARITY_SCORE()

```
similarity_score <- function(mean_matrix = mean_mat){  
  
  ### INPUT CHECKS:  
  
  # check if list  
  if (!is.list(mean_matrix)) {  
    stop("Input must be a list (output from mean_matrix function).")  
  }  
  
  # correct element names  
  required_names <- c("matrices", "score")  
  if (!all(required_names %in% names(mean_matrix))) {  
    stop("Input list is missing 'matrices' or 'score'.  
        Did you provide the correct object?")  
  }  
  
  # check score is a matrix  
  if (!is.matrix(mean_matrix$score)) {  
    stop("The 'score' element must be a matrix.")  
  }  
  
  ### FUNCTION:  
  
  # get score matrix  
  mean_matrix_score <- mean_matrix$score  
  
  # transforms into vector
```



```
result_vector <- as.vector(mean_matrix_score)

# keeps only non NA values
result_vector <- abs(result_vector[!is.na(result_vector)])

# finds squared distance from one
return(mean((1 - result_vector)**2))}
```

Appendix M

SIMULATE_MVN()

```
simulate_mvn <- function(n = 100,  
                          variance = 0.4,  
                          desired_k = 3,  
                          start_seed = 123){  
  
  ### INPUT CHECKS:  
  
  # n is an int  
  if (n %% 1 != 0 || n <= 0) {  
    stop("The number of observations, n, must be a positive integer.")}  
  
  # variance is greater than 0  
  if (variance <= 0) {  
    stop("Variance must be greater than 0.")}  
  
  # desired_k is an positive int  
  if (desired_k %% 1 != 0 || desired_k < 1) {  
    stop("The desired k value must be a positive integer.")}  
  
  # desired_k is 1  
  if (desired_k == 1) {  
    warning("The desired k value is 1 (one cluster).")}
```

```

# start seed is a positive int
if (start_seed %% 1 != 0 || start_seed < 0) {
  stop("The seed value must be an integer.")}

### FUNCTION:
set.seed(start_seed)

# creating covariance matrix
cov_matrix <- diag(2) * variance

# obs per cluster
n_per_cluster <- ceiling(n / desired_k)

# distance between each cluster center
separation <- 3

# create grid coordinates for means
grid_coords <- expand.grid(x = seq(0,
                                by = separation,
                                length.out = ceiling(sqrt(desired_k))),
                           y = seq(0,
                                by = separation,
                                length.out = ceiling(sqrt(desired_k))))

# pick first k grid points
centers <- grid_coords[1:desired_k, ]

```

```

# generate data
sim_data <- do.call(rbind, lapply(1:desired_k, function(k){

  # determine mean
  mu <- c(centers$x[k], centers$y[k])

  # generate clusters
  MASS::mvrnorm(n = n_per_cluster,
                mu = mu,
                Sigma = cov_matrix))}) |>
  as.data.frame() |>
  setNames(c("X1", "X2"))

sim_data}

```